

SemiticGPT

A Practical Guide to Training Multilingual Language Models

From Theory to a Working 3-Billion Parameter Model
for Hebrew, Arabic, Persian, and English

Ronnen Slasky

Independent Researcher `ronnen.slasky@gmail.com`

Code: github.com/fatherRonnen/semitic-gpt · Model: huggingface.co/Slasky/SemiticGPT-3B

April 2026

Who This Guide Is For

This document is for **technical people who are not AI specialists** but want to understand how language models are actually built. If you can read code, understand basic statistics, and have heard of “GPT” but are not sure what happens inside one, this guide is for you.

We walk through every step of building SemiticGPT—a 3.14-billion parameter language model that understands Hebrew, Arabic, Persian, and English—from the fundamental concepts to the specific code that made it work. Every section links to the actual artifacts in our [GitHub repository](https://github.com/fatherRonnen/semitic-gpt), so you can inspect, modify, and reproduce each step.

What you will learn:

- What language models actually are and how they work
- The difference between model architectures and why we chose ours
- How to build a tokenizer for non-Latin scripts
- How to collect and prepare multilingual training data
- How to validate your approach cheaply before committing to expensive training
- How to train a multi-billion parameter model on cloud GPUs
- How to fine-tune and evaluate the result
- What went wrong along the way and what we learned from it

What you will need: Familiarity with Python, basic command-line usage, and access to cloud GPU instances (AWS, GCP, or similar). No prior machine learning experience is assumed.

Contents

1	What Is a Language Model?	5
1.1	The Core Idea	5
1.2	From Words to Numbers: Tokens	5
1.3	How a Language Model Generates Text	6
1.4	What Makes This Hard for Multiple Languages?	6
1.5	The Two Phases of Building a Language Model	7
1.6	How Big Is 3 Billion Parameters?	7
1.7	What This Guide Will Build	8
2	Choosing an Architecture: Encoders, Decoders, and Why It Matters	8
2.1	The Transformer: A Brief History	8
2.2	The Three Families	9
2.2.1	Encoder-Only Models (BERT family)	9
2.2.2	Decoder-Only Models (GPT family)	9
2.2.3	Encoder-Decoder Models (T5 family)	10
2.3	Side-by-Side Comparison	11
2.4	Why We Chose Decoder-Only	11
2.5	Inside the Decoder: Building Blocks	11
2.5.1	Self-Attention: Which Words Matter?	12
2.5.2	Positional Encoding: RoPE	13
2.5.3	Feed-Forward Network: SwiGLU	13
2.5.4	Normalization: RMSNorm	13
2.5.5	Residual Connections	13
2.6	SemiticGPT's Specific Configuration	14
2.7	How Parameters Add Up	14
2.8	Summary: What You Need to Decide	15
3	Building a Multilingual Tokenizer	15
3.1	Why Tokenization Matters	15
3.2	How BPE Tokenization Works	16
3.2.1	Step 1: Start with Characters	16
3.2.2	Step 2: Count Pairs	17
3.2.3	Step 3: Merge the Most Common Pair	17
3.2.4	Step 4: Repeat	17
3.3	The Critical Design Choice: Training Data Balance	17
3.4	SentencePiece: The Implementation	18
3.5	Evaluating Your Tokenizer	19
3.5.1	Fertility (Tokens per Word)	19
3.5.2	Bytes per Token	19
3.6	Practical Impact: A Concrete Example	19
3.7	The Specialization Trade-off	20
3.8	Summary: Building Your Own Tokenizer	20
4	Collecting and Preparing Training Data	21
4.1	How Much Data Do You Need?	21
4.2	Where to Find Multilingual Text	21
4.2.1	Wikipedia	21
4.2.2	OSCAR	22

4.2.3	News Corpora	22
4.2.4	Government Documents	22
4.2.5	Parallel Corpora (for Translation)	22
4.3	Our Data Distribution: The Anchor Language Strategy	22
4.4	Data Cleaning: Turning Raw Text into Training Data	23
4.4.1	Deduplication (MinHash)	23
4.4.2	Language Identification	23
4.4.3	Perplexity-Based Quality Filtering	24
4.4.4	Additional Cleaning	24
4.5	Tokenization: From Clean Text to Training Files	24
4.6	Data Manifests: Tracking What You Have	24
4.7	Summary: The Data Pipeline	25
5	Proxy Experiments: Testing Before Committing	26
5.1	The Idea	26
5.2	What We Tested	26
5.2.1	1. Learning Rate Schedule	26
5.2.2	2. Optimizer Choice	27
5.2.3	3. Peak Learning Rate	27
5.2.4	4. Training Duration (Steps)	27
5.2.5	5. Weight Decay	27
5.2.6	6. Stochastic Weight Averaging (SWA)	27
5.3	The Proxy Architecture	28
5.4	What We Found	28
5.5	Lessons for Your Own Proxy Experiments	29
6	Pretraining: Teaching the Model to Read	29
6.1	What Happens During Pretraining	29
6.2	Distributed Training with FSDP	30
6.2.1	How FSDP Works	30
6.2.2	Mixed Precision: bfloat16	31
6.3	Training Configuration	31
6.4	Monitoring Training	32
6.4.1	Training Loss	32
6.4.2	Validation Loss and BPB	32
6.4.3	Tokens Per Second (TPS)	33
6.4.4	GPU Memory and Utilization	33
6.5	Checkpointing: Saving Your Progress	33
6.6	Summary: Pretraining Checklist	34
7	Supervised Fine-Tuning: Teaching the Model to Work	34
7.1	What SFT Does	34
7.2	Our SFT Data	34
7.2.1	Sentiment Classification (18,980 samples)	35
7.2.2	Translation Pairs (18,000 samples)	35
7.3	The Importance of Data Quality	35
7.4	SFT Training Configuration	36
7.5	The English-Only SFT Control Experiment	36
7.6	SFT Output	36
7.7	Summary: SFT in Practice	37

8	Evaluation: Measuring What Your Model Learned	37
8.1	Why Evaluation Is Harder Than It Looks	37
8.2	Our Evaluation Methods	38
8.2.1	Method 1: Logprob Classification	38
8.2.2	Method 2: Generative Classification	38
8.2.3	Method 3: Cross-Lingual Retrieval	39
8.2.4	Additional Evaluation: Belebele Reading Comprehension	39
8.3	Evaluation Set Construction	40
8.4	Interpreting Results: What Counts as “Good”?	40
8.5	Summary: Evaluation Principles	41
9	Results: What SemiticGPT Can Do	41
9.1	Experiment A: Does Multilingual SFT Work?	41
9.2	Experiment B: Cross-Lingual Transfer Analysis	43
9.2.1	The Pretrained Model Has Cross-Lingual Representations	43
9.2.2	A Language Family Hierarchy	43
9.2.3	The Representation–Task Gap	44
9.2.4	The Retrieval–SFT Trade-off	44
9.3	Experiment C: Tokenizer Efficiency	44
9.4	Belebele: Where the Model Fails	44
9.5	Summary of Results	45
10	What Went Wrong: Data Bugs, Failed Runs, and Hard Lessons	45
10.1	Lesson 1: The Hebrew Label Inversion	45
10.2	Lesson 2: The Arabic Sorting Artifact	46
10.3	Lesson 3: Data Leakage	46
10.4	Lesson 4: Translation Data Quality	47
10.5	Lesson 5: The Failed 1B Training Run	47
10.6	Lesson 6: Optimizer Instability	47
10.7	Lesson 7: Architecture Mismatch	47
10.8	Lesson 8: Instance Type Mistakes	48
10.9	The Meta-Lesson	48
10.10	Verification Checklist	48
11	Practitioner’s Checklist: Building Your Own Multilingual Model	49
11.1	Phase 1: Define Your Scope (1–2 days)	49
11.2	Phase 2: Build the Tokenizer (1–2 days)	49
11.3	Phase 3: Collect and Prepare Data (1–2 weeks)	50
11.4	Phase 4: Run Proxy Experiments (2–3 days)	50
11.5	Phase 5: Pretrain (1–3 days)	50
11.6	Phase 6: Prepare SFT Data (2–3 days)	51
11.7	Phase 7: Supervised Fine-Tuning (1 day)	51
11.8	Phase 8: Evaluate (1–2 days)	52
11.9	Phase 9: Release (1–2 days)	52
11.10	Complete Timeline	53
11.11	The Most Important Advice	53

Chapter 1: What Is a Language Model?

1.1 The Core Idea

A language model is a program that predicts the next word in a sequence. That is, fundamentally, all it does. If you give it the text:

“The capital of France is _____”

a good language model will assign a high probability to “Paris” and a low probability to “banana.” This single capability—predicting what comes next—turns out to be extraordinarily powerful. A model that is good at predicting the next word must, implicitly, understand grammar, facts, reasoning, and even style. It does not “know” these things the way a human does, but it has learned statistical patterns that approximate them.

Key Idea

Language models do not *understand* language in the human sense. They learn statistical patterns over enormous amounts of text. But those patterns are rich enough to produce behavior that looks remarkably like understanding. The philosophical question of whether this constitutes “real” understanding is fascinating but outside the scope of this guide.

1.2 From Words to Numbers: Tokens

Computers work with numbers, not words. Before a language model can process text, it must be converted into a sequence of numbers. This conversion happens through a **tokenizer**—a program that breaks text into smaller units called **tokens** and assigns each one a number.

Tokens are not always whole words. Depending on the tokenizer, a token might be:

- A common word: “the” → token 1234
- Part of a word: “un” + “believe” + “able” → tokens [567, 891, 234]
- A single character: for rare scripts, each letter might be its own token
- A special marker: <s> (start of text) → token 1

Why not just use whole words? Because languages have enormous vocabularies. English alone has hundreds of thousands of words, and when you add Hebrew, Arabic, and Persian, the number explodes. A tokenizer that assigned one number per word would need a massive vocabulary table, and most entries would be rarely used.

Instead, modern tokenizers use **subword algorithms** (we use one called BPE, which we will explain in Chapter 3) that find a balance: common words get their own token, while rare words are broken into reusable pieces. This keeps the vocabulary manageable (our model uses 32,000 tokens) while still being able to represent any text.

Practical Tip

The size of the vocabulary is a design choice with real consequences. Too small (say, 1,000 tokens) and every word gets chopped into many pieces, making text “expensive” to process. Too large (say, 1,000,000 tokens) and the model wastes capacity storing embeddings for tokens it rarely sees. For multilingual models covering 4–6 languages, 32,000–65,000 tokens is a common sweet spot.

1.3 How a Language Model Generates Text

Once text is tokenized into numbers, the language model processes the sequence and outputs a **probability distribution** over the entire vocabulary—a list of 32,000 numbers (one per token) that says how likely each token is to come next.

For example, given “The capital of France is”:

Token	Probability
Paris	0.85
the	0.03
Lyon	0.02
a	0.01
... (31,996 other tokens)	...
banana	0.000001

Table 1: Simplified probability distribution. The model assigns high probability to “Paris” and near-zero probability to “banana.”

To generate text, we **sample** from this distribution—pick one token (usually one of the top candidates), add it to the sequence, and repeat. This is why language model output is often slightly different each time: there is randomness in the sampling step.

Key Idea

A language model produces one token at a time. To generate a paragraph, it runs hundreds of times in a loop, each time taking everything generated so far as input and predicting the next token. This is called **autoregressive generation**: the model’s own output becomes part of its input for the next step.

1.4 What Makes This Hard for Multiple Languages?

Building a language model for English is “easy” (relatively speaking) because:

1. **Abundant data:** Trillions of English words are available online.
2. **Established tokenizers:** Existing tokenizers like Llama-2’s are optimized for English.
3. **Single script:** English uses only Latin characters (A–Z).

Our challenge is harder for several reasons:

Multiple scripts. Hebrew uses its own alphabet (alef-bet), Arabic uses Arabic script (which Persian also uses, but with four extra characters), and English uses Latin. These scripts have different character sets, write in different directions (Hebrew and Arabic are right-to-left), and have different rules for how letters connect.

Rich morphology. Hebrew and Arabic are **morphologically rich**—a single root of three consonants can generate dozens of words through patterns. The Hebrew root K-T-V (write) produces: *katav* (he wrote), *kotev* (he writes), *miktav* (a letter), *ktuva* (written), and many more. This means these languages have far more distinct word forms than English, making tokenization harder.

Limited data. While English has trillions of tokens available, Hebrew has perhaps tens of billions and Arabic somewhat more. Persian falls somewhere in between. This means we cannot simply throw more data at the problem—we must be efficient.

Cross-lingual relationships. Hebrew and Arabic are both Semitic languages—they share a common ancestor and have similar grammatical structures. Persian is unrelated (it is Indo-European, like English) but uses Arabic script and has many Arabic loanwords. We want our model to exploit these relationships: if it learns something about Hebrew grammar, can it reuse that knowledge for Arabic?

1.5 The Two Phases of Building a Language Model

Building a useful language model happens in two distinct phases:

1. **Pretraining:** Feed the model billions of tokens of raw text. It learns language structure, facts, and cross-lingual patterns. After this phase, the model can predict text fluently but does not know how to follow instructions or perform specific tasks. Think of this as a well-read person who has never been asked a question.
2. **Supervised Fine-Tuning (SFT):** Show the model thousands of examples of the behavior you want—question-answer pairs, classification examples, translation pairs. This teaches the model to follow instructions and perform specific tasks. Think of this as training the well-read person to be a useful assistant.

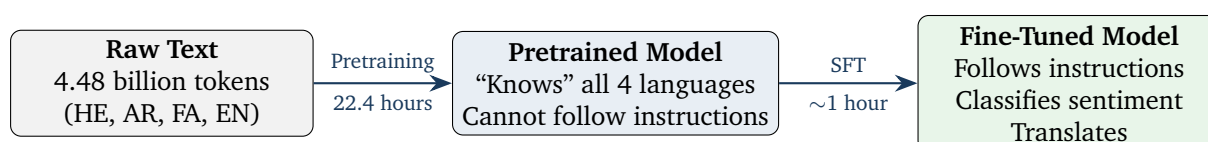


Figure 1: The two-phase process. Pretraining is expensive and teaches general language knowledge. Fine-tuning is cheap and teaches specific behaviors.

An important asymmetry: pretraining is computationally expensive (ours took 22.4 hours on a machine with eight high-end GPUs), while fine-tuning is cheap (about one hour on a single smaller GPU). This means you pretrain once and can fine-tune many times for different tasks.

1.6 How Big Is 3 Billion Parameters?

Our model has 3.14 billion **parameters**—the adjustable numbers that the model learns during training. To give some intuition:

- A simple linear regression has 2 parameters (slope and intercept).
- A small neural network for image recognition might have 1 million parameters.
- GPT-2 (2019) had 1.5 billion parameters.
- **SemiticGPT has 3.14 billion parameters.**
- GPT-3 (2020) had 175 billion parameters.
- GPT-4 (2023) is estimated to have over 1 trillion parameters.

At 3 billion parameters, our model is small by current frontier standards but large enough to learn genuine language understanding. It cannot write essays as well as GPT-4, but it can classify sentiment, align translations, and demonstrate cross-lingual transfer—the phenomena we wanted to study.

Each parameter is stored as a `bf16` number (2 bytes), so the full model weighs about 6.3 GB on disk. It fits comfortably in the memory of a single modern GPU.

1.7 What This Guide Will Build

Over the following chapters, we will build SemiticGPT step by step:

Ch.	Topic	Output	Repo Path
2	Architecture	Model design decisions	pretrain/config/
3	Tokenizer	32K multilingual tokenizer	tokenizer/
4	Data	4.48B token corpus	data/
5	Proxy Experiments	Validated training recipe	proxy/
6	Pretraining	Base model (12.5 GB)	pretrain/
7	Fine-Tuning	Task-capable model (6.3 GB)	sft/
8	Evaluation	Performance measurements	eval/
9	Results	What we found	paper/v2/
10	Lessons	What went wrong	—
11	Checklist	Step-by-step recipe	—

Table 2: Guide roadmap. Each chapter produces a concrete artifact that feeds into the next.

Repository Reference

All code referenced in this guide is available at:

<https://github.com/fatherRonnenn/semitic-gpt>

Model weights are available at:

<https://huggingface.co/Slasky/SemiticGPT-3B>

Chapter 2: Choosing an Architecture: Encoders, Decoders, and Why It Matters

Before you can train a language model, you must choose its architecture—the blueprint that determines how information flows through the model. This choice affects what your model can do, how it is trained, and how it is used. The dominant architecture in modern NLP is the **transformer** [Vaswani et al., 2017], but there are three major variants, and they are suited to different tasks.

2.1 The Transformer: A Brief History

Before 2017, most language models used architectures called **recurrent neural networks (RNNs)** that processed text one word at a time, left to right. This was slow (you could not parallelize it) and caused problems with long sequences (by the time the model reached word 500, it had largely “forgotten” word 1).

The transformer, introduced by Vaswani et al. [2017], solved both problems with a mechanism called **attention**: instead of reading text sequentially, the model looks at all words simultaneously and learns which words are relevant to which other words. This is massively parallelizable on GPUs and handles long-range dependencies naturally.

Key Idea

The key insight of the transformer is **attention**: for each word in the input, the model computes a weighted combination of all other words, where the weights reflect relevance. “The cat sat on the mat because *it* was tired”—to understand “it,” the model attends strongly to “cat” and weakly to “mat,” even though “mat” is closer.

The original transformer had two halves: an **encoder** (which reads the input) and a **decoder** (which generates the output). Subsequent research found that you could use just one half and still get excellent results, leading to three families:

2.2 The Three Families

2.2.1 Encoder-Only Models (BERT family)

How they work: The model reads the *entire* input at once, in both directions. Every word can attend to every other word, including words that come *after* it. This is called **bidirectional** attention.

Training method: Mask out random words and ask the model to predict them. For example:

The [MASK] sat on the mat → predict “cat”

This is called **masked language modeling (MLM)**. The model learns by filling in blanks, which forces it to understand context from both sides.

What they are good at:

- **Classification:** Is this review positive or negative? What language is this?
- **Named entity recognition:** Which words are person names, locations, organizations?
- **Semantic similarity:** Are these two sentences saying the same thing?
- **Information retrieval:** Find the most relevant document for this query.

What they cannot do: Generate text. Because they see the whole input at once, they have no mechanism for producing text one token at a time. You cannot ask BERT to write a paragraph.

Famous examples: BERT [Devlin et al., 2019], RoBERTa, XLM-R [Conneau et al., 2020], mBERT.

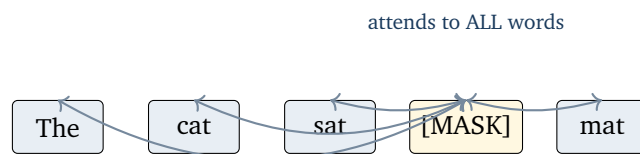


Figure 2: Encoder-only (BERT-style): The masked token attends to all other tokens, both left and right. This bidirectional context makes encoders excellent at understanding but unable to generate.

2.2.2 Decoder-Only Models (GPT family)

How they work: The model reads text left to right. Each word can only attend to words that came *before* it (and itself). It never “peeks ahead.” This is called **causal** or **autoregressive** attention.

Training method: Predict the next token. Given “The cat sat,” predict “on.” Given “The cat sat on,” predict “the.” This is called **causal language modeling (CLM)**. The model processes the entire sequence in one pass, but each position can only see preceding positions.

What they are good at:

- **Text generation:** Write articles, code, poetry, translations.
- **Chat and instruction following:** Answer questions, follow complex prompts.
- **Few-shot learning:** Give a few examples in the prompt and the model generalizes.
- **Classification** (with a trick): Frame it as text generation—“Is this positive or negative? Answer:” and check what the model generates.

What they are less good at: Pure classification tasks where bidirectional context would help. However, in practice, large enough decoder models can match or beat encoder models even on classification by using the generation trick described above.

Famous examples: GPT [Radford et al., 2018], GPT-2 [Radford et al., 2019], GPT-3 [Brown et al., 2020], GPT-4, Llama, Mistral, **SemiticGPT (this project)**.

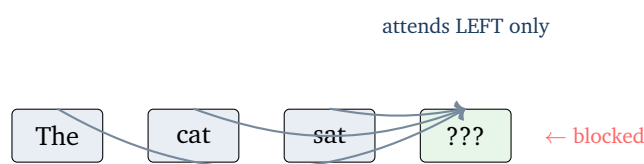


Figure 3: Decoder-only (GPT-style): Each position attends only to previous positions. This constraint enables generation: the model never sees future tokens, so it can produce them one by one.

2.2.3 Encoder-Decoder Models (T5 family)

How they work: Two separate transformer stacks. The encoder reads the full input bidirectionally (like BERT). The decoder generates output autoregressively (like GPT), but can also attend to the encoder’s representations. This is the original transformer architecture.

Training method: Varies, but T5 [Raffel et al., 2020] uses a **span corruption** objective: mask out random spans of text and ask the model to generate just the missing pieces.

What they are good at:

- **Translation:** Read source language (encoder), generate target language (decoder).
- **Summarization:** Read long document (encoder), generate summary (decoder).
- **Structured output:** Any task that maps one text to another.

Trade-off: More complex, harder to scale, and the encoder and decoder must be sized relative to each other. In practice, decoder-only models have won the scaling race because they are simpler and more flexible.

Famous examples: T5 [Raffel et al., 2020], mT5, BART, mBART.

2.3 Side-by-Side Comparison

Property	Encoder (BERT)	Decoder (GPT)	Enc-Dec (T5)
Attention direction	Bidirectional	Left-only	Both
Can generate text?	No	Yes	Yes
Can classify?	Yes	Yes (trick)	Yes
Training objective	Fill blanks	Predict next	Fill spans
Scaling simplicity	Medium	Simple	Complex
Parameter efficiency	High	Medium	Lower
Dominant use today	Embeddings	Chat, gen.	Translation

Table 3: Comparison of the three transformer families. Decoder-only models dominate current large-scale development due to their simplicity and flexibility.

2.4 Why We Chose Decoder-Only

For SemiticGPT, we chose the decoder-only architecture. Here is why:

1. **Flexibility.** A decoder-only model can generate text *and* classify. An encoder-only model can only classify. Since we wanted to study both generation and classification across languages, the decoder gives us more options with one model.
2. **Simplicity.** Decoder-only models have one transformer stack, one training objective (predict the next token), and one inference mode. This means fewer hyperparameters to tune, simpler code, and fewer things that can go wrong. When you are training a model from scratch on a budget, simplicity matters.
3. **Ecosystem.** The open-source ecosystem in 2025–2026 is overwhelmingly decoder-only. Llama, Mistral, SmolLM3 [SmolLM3, 2025]—the models with the best training recipes, the most community support, and the most tooling are all GPT-style decoders. Swimming with the current means we can reuse proven techniques.
4. **Cross-lingual transfer research.** We specifically wanted to test whether a decoder model develops cross-lingual representations during pretraining. Encoder models like XLM-R [Conneau et al., 2020] are already known to do this well. Whether *decoder* models do it at small scale was an open question.
5. **SFT compatibility.** The entire instruction-tuning and chat paradigm is built around decoder models. If you want your model to follow instructions (“Classify this as positive or negative”), decoder-only is the natural fit.

Warning

Choosing decoder-only means accepting a trade-off: for *pure* classification tasks (no generation needed), an encoder model of the same size will usually be more accurate. If your only goal is building a sentiment classifier, fine-tuning XLM-R would be more efficient. We chose the decoder because our goals were broader.

2.5 Inside the Decoder: Building Blocks

Now that we have chosen the decoder-only architecture, let us look at what is inside. A decoder is a stack of identical **layers** (we use 26), each containing the same three components:

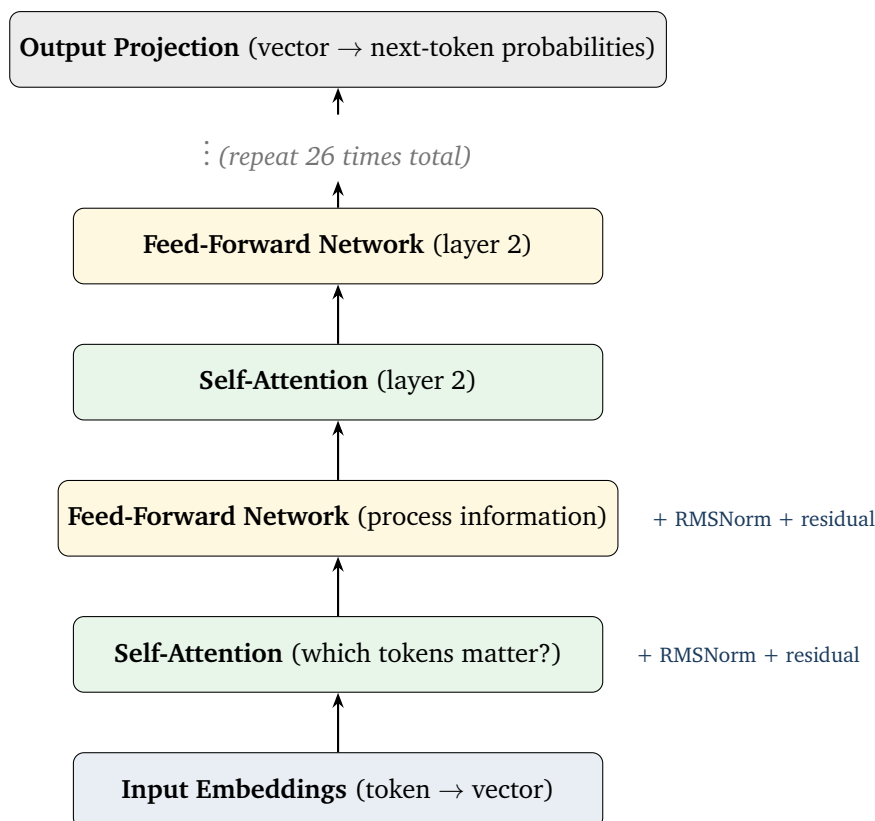


Figure 4: SemiticGPT’s architecture: 26 identical layers, each with self-attention and a feed-forward network, plus normalization and residual connections. Input tokens are converted to vectors; the final layer converts vectors back to token probabilities.

2.5.1 Self-Attention: Which Words Matter?

Self-attention is the mechanism that lets each token “look at” other tokens in the sequence and decide how much to pay attention to each one. Here is the intuition:

Consider the sentence: “The bank by the river was flooded.” To understand “bank,” the model must attend to “river” and “flooded” (which tell it this is a riverbank, not a financial bank). Self-attention learns these relevance patterns automatically from data.

Mechanically, each token is transformed into three vectors:

- **Query (Q):** “What am I looking for?”
- **Key (K):** “What do I contain?”
- **Value (V):** “What information do I provide?”

The attention score between two tokens is the dot product of one token’s Query with the other’s Key. High scores mean high relevance, and the output is a weighted sum of all Values.

SemiticGPT uses **24 attention heads**—24 independent sets of Q/K/V, each with a head dimension of 128. Different heads can learn different types of relationships (one might learn syntactic structure, another might learn semantic similarity). Their outputs are concatenated: $24 \times 128 = 3072$, which equals our hidden dimension.

Key Idea

The **causal mask** is what makes a decoder a decoder. During attention, we add a mask that sets all attention scores to $-\infty$ for future positions. This means token 5 can attend to tokens 1–5 but not tokens 6, 7, 8, etc. This constraint is what enables autoregressive generation.

2.5.2 Positional Encoding: RoPE

Attention is **permutation-invariant**—without positional information, the model cannot tell the difference between “dog bites man” and “man bites dog.” We need a way to encode *where* each token is in the sequence.

SemiticGPT uses **Rotary Position Embedding (RoPE)** [Su et al., 2021], which encodes position by rotating the Query and Key vectors by an angle proportional to their position. Tokens that are close together have similar rotations, which naturally biases attention toward nearby tokens while still allowing long-range attention.

RoPE has two practical advantages over older methods: it handles sequences of any length (up to our maximum of 2,048 tokens), and it encodes *relative* position (how far apart two tokens are) rather than absolute position, which generalizes better.

2.5.3 Feed-Forward Network: SwiGLU

After attention determines *which* information is relevant, the feed-forward network (FFN) *processes* that information. Each layer’s FFN is a small two-layer neural network that operates on each token independently.

SemiticGPT uses **SwiGLU** [Shazeer, 2020], a modern activation function that combines a smooth gating mechanism with a nonlinear transformation. The practical effect: better training stability and slightly better final performance compared to the older ReLU activation, at the cost of about 50% more parameters in the FFN layer.

2.5.4 Normalization: RMSNorm

Deep networks are unstable—values can grow exponentially large or vanish to zero as they pass through many layers. **Normalization** keeps values in a reasonable range after each sub-layer.

SemiticGPT uses **RMSNorm** (Root Mean Square Normalization) [Zhang and Sennrich, 2019], applied *before* each attention and FFN block (called “pre-norm”). RMSNorm is simpler and faster than the more common LayerNorm, and works equally well in practice. It normalizes by dividing by the root-mean-square of the values, with a learned scaling parameter.

2.5.5 Residual Connections

Each sub-layer (attention and FFN) has a **residual connection**: the sub-layer’s input is *added* to its output. So if the input is x and the sub-layer computes $f(x)$, the output is $x + f(x)$.

This simple trick is essential for training deep networks. It creates a “highway” for gradients to flow backward through many layers during training, preventing the vanishing gradient problem. Without residual connections, training a 26-layer network would be very difficult.

2.6 SemiticGPT’s Specific Configuration

Parameter	Value	Why this choice
Parameters	3.14B	Large enough for meaningful language understanding; small enough to train on one node and fit on one GPU for inference.
Layers	26	Standard depth for 3B-class models. More layers = more capacity for abstract reasoning.
Hidden dim	3,072	Determines the “width” of information flowing through the model. $24 \text{ heads} \times 128 \text{ dim} = 3,072$.
Attention heads	24	Each head can specialize in different types of relationships. 24 is standard for this model size.
Head dimension	128	The size of each attention head. 128 is the modern default (older models used 64).
Vocab size	32,000	Balances coverage of 4 languages against embedding table size. See Chapter 3.
Max sequence	2,048	How much text the model can process at once. Sufficient for most tasks; longer sequences cost quadratically more memory.
Position encoding	RoPE	Handles variable-length sequences and relative positions naturally.
Activation	SwiGLU	Better than ReLU for training stability. Standard in modern models.
Normalization	RMSNorm	Simpler and faster than LayerNorm. Applied before each sub-layer (pre-norm).

Table 4: SemiticGPT architecture choices and rationale. See `pretrain/config/training_config.json` in the repository.

Repository Reference

Architecture configuration: `pretrain/config/training_config.json`
Model implementation: `pretrain/scripts/train_multilingual_3b_fsd.py`

The model definition starts around line 60 of the FSDP training script. The architecture is defined in pure PyTorch—no framework dependencies. Key classes: `Block` (one transformer layer), `GPT` (the full model with all layers, embeddings, and output projection).

2.7 How Parameters Add Up

Where do 3.14 billion parameters come from? Here is a rough breakdown:

Component	Parameters	% of Total
Token embeddings ($32,000 \times 3,072$)	98M	3.1%
Attention QKV projections ($26 \times 3 \times 3,072 \times 3,072$)	738M	23.5%
Attention output projections ($26 \times 3,072 \times 3,072$)	246M	7.8%
FFN (SwiGLU, 3 matrices per layer)	1,769M	56.3%
RMSNorm parameters ($26 \times 2 + 1$ layers)	<1M	<0.1%
Output projection (tied with embeddings)	0	0%
Total	~3,140M	100%

Table 5: Parameter breakdown. The feed-forward networks account for over half of all parameters, which is typical for transformer models. The attention mechanism accounts for about a third.

Practical Tip

Notice that the token embeddings (the table mapping token IDs to vectors) are **shared** with the output projection (the table mapping vectors back to token probabilities). This is called **weight tying** and saves 98M parameters with no quality loss—the intuition is that the “meaning” of a token should be the same whether the model is reading it or producing it.

2.8 Summary: What You Need to Decide

If you are building your own language model, the architecture decisions boil down to:

1. **Encoder, decoder, or encoder-decoder?** For most modern use cases involving generation or instruction-following, decoder-only is the default choice.
2. **How many parameters?** Constrained by your GPU budget. Aim for the largest model you can afford to train to completion—an undertrained large model is worse than a well-trained smaller one.
3. **Depth vs. width?** For a given parameter budget, modern practice favors deeper (more layers) over wider (larger hidden dimension), but the difference is small.
4. **Vocabulary size?** 32K–64K for multilingual models. See Chapter 3.
5. **Everything else:** Use the modern defaults (RoPE, SwiGLU, RMSNorm, pre-norm) unless you have a specific reason not to. These choices are battle-tested and well-supported in existing code.

Chapter 3: Building a Multilingual Tokenizer

Before a language model can process any text, that text must be converted into numbers. The component that does this is the **tokenizer**, and for a multilingual model, it is one of the most consequential design decisions you will make. A bad tokenizer can cripple your model before training even begins.

3.1 Why Tokenization Matters

Consider a simple thought experiment. Suppose your tokenizer was designed for English and you feed it a Hebrew sentence. The tokenizer has never seen Hebrew characters, so it falls

back to encoding each character—or even each *byte*—as a separate token. A five-word Hebrew sentence might become 40 tokens, while the equivalent English sentence might be 7 tokens.

This has three consequences:

1. **Wasted context window.** If your model can process 2,048 tokens at a time, the English user gets ~ 300 words of context, while the Hebrew user gets ~ 60 . The Hebrew user is effectively using a much smaller, less capable model.
2. **Slower learning.** The model must process $5\text{--}6\times$ more tokens to see the same amount of Hebrew text. Since training is measured in tokens, Hebrew gets $5\text{--}6\times$ less “learning” per training step.
3. **Harder patterns.** When words are split into individual characters, the model must learn to reassemble them before it can learn word meanings. This is like trying to learn a language by reading one letter at a time through a keyhole.

This is not hypothetical. We measured it. Using Llama-2’s English-optimized tokenizer on our languages:

Language	Llama-2 (tokens/word)	Ours (tokens/word)
English	1.51	1.54
Hebrew	3.91	1.34
Arabic	4.36	2.22
Persian	4.88	1.52

Table 6: Tokens per word (lower is better). Llama-2 fragments Hebrew, Arabic, and Persian into $4\text{--}5\times$ more tokens than necessary. Our balanced tokenizer eliminates this penalty.

Llama-2 needs almost 5 tokens for every Persian word versus 1.5 for ours. That is a $3.2\times$ efficiency gap—meaning our model can process $3.2\times$ more Persian text in the same context window.

Key Idea

The tokenizer determines how “expensive” each language is for your model. An unbalanced tokenizer creates a hidden tax on underrepresented languages, making them harder to learn and less capable at inference time. For a multilingual model, this tax must be eliminated.

3.2 How BPE Tokenization Works

Our tokenizer uses an algorithm called **Byte Pair Encoding (BPE)** [Sennrich et al., 2016], implemented in Google’s **SentencePiece** library [Kudo and Richardson, 2018]. Here is how it works, step by step:

3.2.1 Step 1: Start with Characters

Begin with every unique character in your training data as a separate token. For four languages, this might be ~ 300 characters: 26 Latin letters (upper and lower), 22 Hebrew letters, 28+ Arabic characters, digits, punctuation, and special symbols.

At this point, every word is split into individual characters:

“hello” $\rightarrow$ [h, e, l, l, o]

3.2.2 Step 2: Count Pairs

Scan the entire training corpus and count how often each adjacent pair of tokens appears. For English text, the pair (t, h) will be very common (from “the,” “that,” “this,” etc.).

3.2.3 Step 3: Merge the Most Common Pair

Take the most frequent pair and merge it into a new token. If (t, h) is the most common, create a new token th. Now every occurrence of t followed by h becomes the single token th.

3.2.4 Step 4: Repeat

Go back to Step 2 with the updated tokens. The next most common pair might be (th, e), creating the token the. Continue until you reach your desired vocabulary size (32,000 in our case).

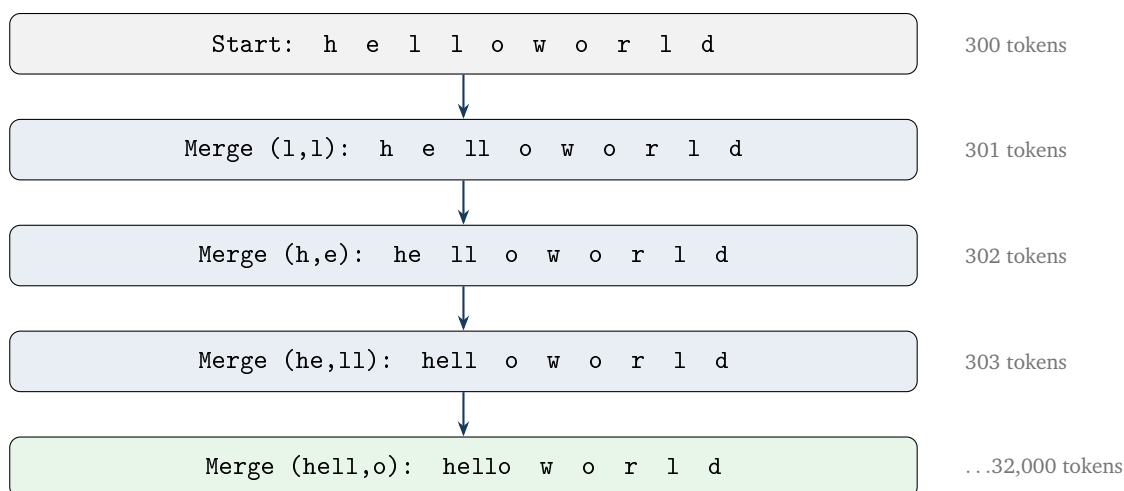


Figure 5: BPE builds tokens bottom-up by repeatedly merging the most frequent pair. Common words like “hello” eventually become single tokens, while rare words remain split into subwords.

The beauty of BPE is that it is **data-driven**: it automatically discovers the most useful subwords for whatever languages are in the training data. Common words become single tokens, common prefixes and suffixes become tokens, and truly rare strings are broken into small pieces.

3.3 The Critical Design Choice: Training Data Balance

Here is where multilingual tokenization gets interesting. BPE learns its vocabulary from the training data you give it. If you feed it 90% English and 10% Hebrew, it will create a vocabulary that is excellent for English but fragments Hebrew badly—because English pairs will always be more common and get merged first.

Our approach: We train the tokenizer on an **equally balanced** sample—25% from each language. This ensures that Hebrew character pairs get merged just as aggressively as English pairs, producing a vocabulary that works well for all four languages.

Script	Ours	Llama-2	HebrewGPT
Arabic script	46.7%	0.2%	0.4%
Hebrew script	27.8%	0.1%	72.2%
Latin script	24.3%	80.9%	7.0%
Other	1.2%	18.8%	20.4%

Table 7: Vocabulary composition by script. Llama-2 is 81% Latin—almost useless for our target languages. HebrewGPT is 72% Hebrew—great for Hebrew, terrible for Arabic and Persian. Ours balances all three scripts.

Notice that Arabic script tokens (46.7%) outnumber Hebrew tokens (27.8%) in our vocabulary, even though each language gets 25% of the training data. This is because Persian *also* uses Arabic script (with minor additions), so both Arabic and Persian contribute to Arabic-script tokens.

Warning

You might worry that balancing the tokenizer hurts English performance—after all, English gets only 25% of the tokenizer training data instead of nearly 100% in Llama-2. In practice, the penalty is tiny: our tokenizer uses 1.54 tokens per English word versus Llama-2’s 1.51. A 2% overhead on English in exchange for 49–69% improvement on three other languages is an excellent trade.

3.4 SentencePiece: The Implementation

We use Google’s **SentencePiece** library [Kudo and Richardson, 2018] rather than implementing BPE from scratch. SentencePiece adds several practical features:

- **Language-agnostic:** It operates on raw Unicode text, not pre-tokenized words. This is critical for languages like Chinese, Japanese, or Arabic where word boundaries are ambiguous.
- **Byte fallback:** If the tokenizer encounters a character it has never seen, it can fall back to encoding individual bytes. This means the tokenizer can handle *any* text, even in languages not in the training data.
- **Normalization:** SentencePiece applies Unicode normalization (NFKC) to standardize different encodings of the same character. Arabic, for example, has multiple Unicode representations for some letters.
- **Special tokens:** We define special tokens for model control:

<s>	Beginning of sequence
</s>	End of sequence
<pad>	Padding (for batching sequences of different lengths)
< user >	Start of user message (for instruction following)
< assistant >	Start of model response

Repository Reference

Tokenizer training script:	<code>tokenizer/scripts/train_tokenizer.py</code>
Tokenizer configuration:	<code>tokenizer/config/tokenizer_config.json</code>
Fertility analysis results:	<code>tokenizer/artifacts/fertility_report.json</code>
Vocabulary statistics:	<code>tokenizer/artifacts/vocabulary_stats.json</code>

The training script collects balanced samples from each language, trains SentencePiece, and produces the `.model` and `.vocab` files used during pretraining.

3.5 Evaluating Your Tokenizer

Once you have a tokenizer, how do you know it is good? We use two metrics:

3.5.1 Fertility (Tokens per Word)

Fertility measures how many tokens the tokenizer needs per word on average. Lower is better.

- **Fertility = 1.0:** Every word is a single token (ideal but impossible for large vocabularies).
- **Fertility = 1.5:** Most words are one token, some common words need two pieces.
- **Fertility = 4.0:** Most words are split into many pieces (very inefficient).

For a well-designed multilingual tokenizer, you want fertility below 2.0 for all target languages. Our tokenizer achieves this for English (1.54), Hebrew (1.34), and Persian (1.52). Arabic (2.22) is slightly higher due to the language’s morphological complexity.

3.5.2 Bytes per Token

Bytes per token measures how much raw text each token represents. Higher is better.

This metric is useful because different scripts use different numbers of bytes per character. Hebrew characters are 2 bytes in UTF-8, while English characters are 1 byte. Bytes per token normalizes across scripts:

- Our tokenizer: Hebrew = 5.12 bytes/token
- Llama-2: Hebrew = 1.76 bytes/token

Our tokenizer packs $2.9\times$ more Hebrew text into each token.

3.6 Practical Impact: A Concrete Example

Consider two short sentences and how different tokenizers handle them:

Hebrew: “habina hamela’khutit meshane et ha’olam” (Artificial intelligence is changing the world):

- **Ours:** 4 tokens (whole-word segmentation)
- **Llama-2:** 18 tokens (character-level fragmentation)
- **HebrewGPT:** 5 tokens (comparable—Hebrew-optimized)

Arabic: “asimat faransa hia baris” (The capital of France is Paris):

- **Ours:** 5 tokens (whole-word segmentation)

- **Llama-2**: 22 tokens (character-level fragmentation)
- **HebrewGPT**: 19 tokens (character-level fragmentation)

The difference is dramatic: 4–5 tokens versus 18–22. This means our model processes $4\times$ more text per forward pass, learns $4\times$ faster per training step, and provides $4\times$ more context at inference.

3.7 The Specialization Trade-off

You might ask: why not use HebrewGPT’s tokenizer? It achieves 1.26 tokens per Hebrew word—slightly better than ours (1.34). The answer is that HebrewGPT’s tokenizer is a **specialist**: 72% of its vocabulary is Hebrew script. It handles Hebrew beautifully but fragments Arabic (4.15 tokens/word) and Persian (4.51 tokens/word) as badly as Llama-2.

We need a **generalist**: a tokenizer that performs well across all four languages, even if it is not the absolute best for any single one.

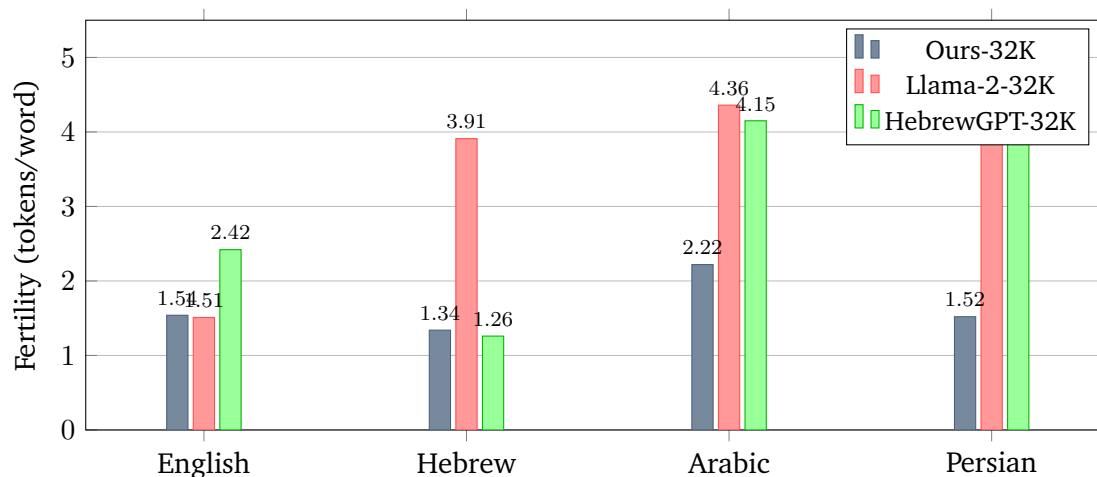


Figure 6: Tokenizer fertility comparison (lower is better). Our tokenizer (blue) is the only one that performs well across all four languages. Llama-2 (red) is good for English only. HebrewGPT (green) is good for Hebrew only.

3.8 Summary: Building Your Own Tokenizer

If you are building a multilingual model:

1. **Do not reuse an English tokenizer.** The efficiency penalty on non-Latin scripts is severe ($3\text{--}5\times$).
2. **Train on balanced data.** Equal language proportions during tokenizer training ensures fair vocabulary allocation.
3. **Use SentencePiece BPE** with byte fallback and NFKC normalization.
4. **Choose 32K–64K vocabulary** for 4–6 languages. Smaller vocabularies under-represent some languages; larger vocabularies waste embedding parameters.
5. **Verify with fertility analysis.** Target ≤ 2.0 tokens per word for all languages. Check the English penalty—it should be minimal ($<5\%$).
6. **Add special tokens** for your intended use case (instruction following, chat, etc.) *before* training the tokenizer, so they get reserved IDs.

Practical Tip

Tokenizer training is fast (minutes, not hours) and does not require a GPU. You can iterate on the tokenizer many times, testing different vocabulary sizes and language balances, before committing to pretraining. This is one of the cheapest experiments you can run, so do not skimp on it.

Chapter 4: Collecting and Preparing Training Data

A language model is only as good as the text it learns from. This chapter covers where to find multilingual training data, how to clean it, how to balance it across languages, and the practical decisions that shaped our 4.48-billion-token corpus.

4.1 How Much Data Do You Need?

The relationship between model size and training data is governed by **scaling laws**—empirical findings about how much data a model of a given size needs to train optimally. The most cited rule of thumb comes from the Chinchilla paper (Hoffmann et al., 2022): a model should be trained on roughly $20\times$ its parameter count in tokens.

For our 3.14B parameter model, that suggests:

$$3.14\text{B} \times 20 = 62.8\text{B tokens (optimal)}$$

We trained on 4.48B tokens—about $14\times$ less than the Chinchilla-optimal amount. Why?

1. **Data availability.** For Hebrew, high-quality text simply does not exist at the 60B-token scale. You cannot train on data you do not have.
2. **Budget constraints.** Training on 62B tokens would take roughly $14\times$ longer (and cost $14\times$ more). Our goal was a research model, not a production one.
3. **Diminishing returns at small scale.** Scaling laws assume you are pushing frontier performance. For our research questions about cross-lingual transfer, 4.48B tokens proved sufficient to demonstrate the phenomena.

Key Idea

For low-resource languages, you are almost always **data-limited, not compute-limited**. The bottleneck is finding enough clean text in your target languages, not buying enough GPUs. This makes data collection and quality the most important phase of the project.

4.2 Where to Find Multilingual Text

Our corpus draws from several public sources. Here is what is available and what we used:

4.2.1 Wikipedia

What it is: Free encyclopedia articles in 300+ languages.

Quality: High—edited, factual, formal register.

Size: Small for most languages. Hebrew Wikipedia is about 1.2 GB of raw text; Arabic is about 2.1 GB.

License: CC-BY-SA (free to use with attribution).

Where to get it: <https://dumps.wikimedia.org/>

Wikipedia is a good foundation but nowhere near sufficient on its own. It represents only one style (encyclopedic) and is relatively small.

4.2.2 OSCAR

What it is: A massive multilingual web crawl, filtered by language. Part of the Common Crawl ecosystem.

Quality: Variable—ranges from clean news articles to near-gibberish. Requires filtering.

Size: Large. Hebrew OSCAR is about 8 GB; Arabic is about 12 GB.

License: CC-BY-4.0.

Where to get it: <https://oscar-project.org/>

OSCAR provides the bulk of our training data. The trade-off is volume versus quality: you get a lot of text, but you must invest in cleaning it.

4.2.3 News Corpora

What it is: Collections of news articles from various publications.

Quality: High—professionally written, diverse topics.

Size: Medium. Typically 1–5 GB per language.

License: Varies; often “research only.”

News text provides valuable formal-register content and topical diversity.

4.2.4 Government Documents

For Hebrew specifically, Israeli government publications, Knesset proceedings, and legal texts provide high-quality formal Hebrew. These are often overlooked but are usually publicly available.

4.2.5 Parallel Corpora (for Translation)

For our supervised fine-tuning phase (Chapter 7), we also need **parallel text**—the same content in two languages, sentence-aligned. Our primary source is **OPUS-100** [Zhang et al., 2020b], which provides millions of sentence pairs for many language combinations.

Warning

Not all language pairs are available in parallel corpora. OPUS-100 has English↔Hebrew, English↔Arabic, and English↔Persian, but *not* Hebrew↔Arabic or Hebrew↔Persian directly. This is a common limitation for low-resource language pairs.

4.3 Our Data Distribution: The Anchor Language Strategy

We do not split data equally across languages. Instead, we use an **anchor language** strategy, overrepresenting Hebrew:

Language	Share	~Tokens	Rationale
Hebrew	40%	1.79B	Anchor language—strongest representations
Arabic	20%	0.90B	Semitic sibling to Hebrew
English	20%	0.90B	High-resource bridge language
Persian	20%	0.90B	Typological control (Indo-European)
Total	100%	4.48B	

Table 8: Pretraining data distribution. Hebrew receives $2\times$ the share of other languages to establish strong anchor representations.

Why overrepresent Hebrew? The idea is that by giving one language extra data, the model builds especially strong internal representations for it. If cross-lingual transfer works, those strong Hebrew representations might “pull up” related languages (especially Arabic, which shares Semitic roots) during training. Whether this actually works better than equal distribution is an open question—we did not ablate it—but the strategy is common in multilingual modeling.

Practical Tip

Note that the tokenizer is trained on equal 25% shares, but the *pretraining data* uses 40/20/20/20. These are independent decisions. The tokenizer balance ensures fair vocabulary; the data balance determines how much each language is learned during pre-training.

4.4 Data Cleaning: Turning Raw Text into Training Data

Raw web text is messy. Here are the cleaning steps we apply, and why each matters:

4.4.1 Deduplication (MinHash)

Problem: Web crawls contain enormous amounts of duplicate content—the same article republished on dozens of sites, boilerplate headers/footers, cookie notices repeated on every page.

Solution: **MinHash deduplication** creates a compact “fingerprint” of each document and efficiently finds near-duplicates. Documents that are more than $\sim 80\%$ similar to an existing document are removed.

Why it matters: Duplicate data causes the model to memorize specific texts rather than learning general patterns. It also wastes training compute on content the model has already seen.

4.4.2 Language Identification

Problem: OSCAR and other web corpora are filtered by language, but the filtering is imperfect. A “Hebrew” document might actually be in Russian, or might be mostly English with a few Hebrew words.

Solution: Run a language identification model (such as FastText’s `lid.176.bin`) on each document and discard documents where the detected language does not match the expected language.

Why it matters: Training on mislabeled data confuses the model about what each language looks like.

4.4.3 Perplexity-Based Quality Filtering

Problem: Some documents are technically in the right language but are very low quality—machine-generated text, SEO spam, random word salad, or text extraction artifacts.

Solution: Train a small language model on known-good text (e.g., Wikipedia) and use it to score each document’s **perplexity**—how “surprised” the model is by the text. Discard documents with very high perplexity (the model finds them incomprehensible) or very low perplexity (likely repetitive boilerplate).

Why it matters: Quality filtering has a larger impact on downstream performance than simply adding more data. A smaller clean dataset often beats a larger noisy one.

4.4.4 Additional Cleaning

- **HTML/boilerplate removal:** Strip remaining HTML tags, navigation menus, and page templates.
- **Length filtering:** Remove documents shorter than 50 characters (likely fragments) or extremely long single-paragraph documents (likely extraction errors).
- **Personal information:** Remove email addresses, phone numbers, and other PII where detected.
- **Unicode normalization:** Standardize different Unicode representations of the same character (important for Arabic script, which has multiple forms).

4.5 Tokenization: From Clean Text to Training Files

Once the text is clean, we run it through our tokenizer (Chapter 3) to convert it from human-readable text into sequences of token IDs—integers between 0 and 31,999.

The tokenized data is stored as **binary files** of unsigned 16-bit integers (`uint16`), one per token. This is compact (2 bytes per token) and fast to read during training. A 4.48B-token dataset takes about 8.96 GB on disk in this format.

During training, the data loader reads these binary files and constructs training examples by slicing them into fixed-length sequences of 2,048 tokens. No padding is needed because the binary format is contiguous—the end of one document flows into the beginning of the next, separated by end-of-sequence tokens.

4.6 Data Manifests: Tracking What You Have

With data coming from multiple sources in multiple languages, organization is critical. We maintain **manifests**—JSON files that record exactly what is in the dataset:

- **Source manifest:** For each language, lists every data source with its URL, estimated size, type (encyclopedia, web crawl, news), and license.
- **Mixture specification:** Records the target proportions for each language and the mixing strategy.
- **Training data metadata:** Records the actual token counts after processing, checksums, and creation dates.

This may seem like bureaucratic overhead, but it is essential for reproducibility and debugging. When your model behaves unexpectedly on Arabic (as ours did—see Chapter 10), you need to be able to trace exactly what Arabic data it trained on.

Repository Reference

Data collection scripts:

<code>collect_native_data_v2.py</code>	<code>data/scripts/</code> Main collection pipeline
<code>arabic_collect.py</code>	Arabic-specific collection
<code>farsi_collect.py</code>	Persian-specific collection
<code>collect_translation_data_v2.py</code>	Parallel corpus collection
<code>retokenize_arabic.py</code>	Arabic retokenization after rebalancing

Data manifests:

<code>source_manifest.json</code>	<code>data/manifests/</code> All data sources with URLs and licenses
<code>mixture_spec.json</code>	Language proportions
<code>training_data_metadata.json</code>	Final token counts and checksums

4.7 Summary: The Data Pipeline

Here is the complete pipeline from raw sources to training-ready data:

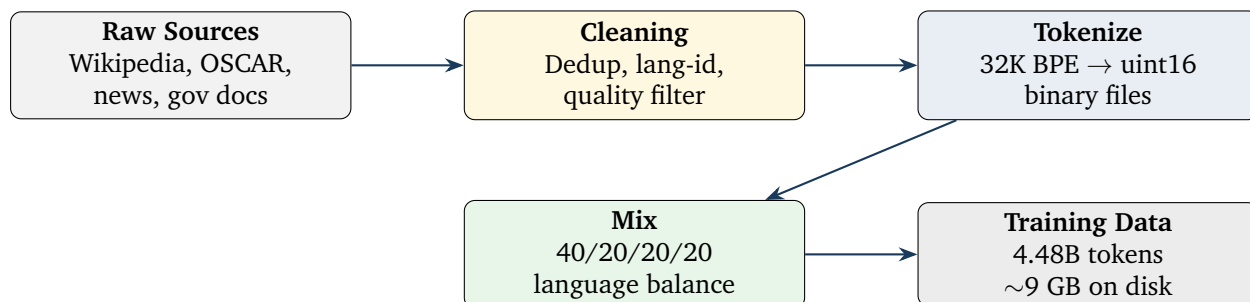


Figure 7: The data pipeline. Each step reduces volume but increases quality. The final output is a compact binary file ready for the training loop.

1. **Collect** raw text from multiple sources per language.
2. **Clean** with deduplication, language ID, and quality filtering.
3. **Tokenize** with your custom multilingual tokenizer.
4. **Mix** according to your target language proportions.
5. **Store** as compact binary files with manifests for tracking.
6. **Verify** token counts, language proportions, and data integrity before training.

Practical Tip

Invest in data quality *before* you start training. Once pretraining begins, it is extremely expensive to restart because you discovered a data problem. We spent roughly one week on data collection and cleaning versus one day on actual pretraining. That ratio is typical and appropriate.

Chapter 5: Proxy Experiments: Testing Before Committing

Pretraining a 3B model costs real money and takes real time. If your training recipe is wrong—bad learning rate, wrong optimizer, broken schedule—you will not discover this until hours or days into the run, by which point you have wasted the entire investment. **Proxy experiments** solve this by testing your recipe at small scale first.

5.1 The Idea

A proxy experiment is a miniature version of your full training run. Instead of training a 3B model on 4.48B tokens (22 hours on 8 GPUs), you train a 110M model on a few hundred million tokens (2–5 minutes on 1 GPU). The hypothesis is that a recipe that works well at small scale will also work well at large scale. This is not always true, but it is true often enough to be extremely useful.

Key Idea

Proxy experiments let you explore 30+ recipe variations in the time it takes to run one full training job. We ran **32 proxy experiments** to select our final training recipe before committing to the 3B pretraining run. Total cost: roughly 16 GPU-hours on a single L40S. The full run cost 180 GPU-hours on 8×H100. The proxy experiments were less than 1% of the cost but eliminated the most dangerous recipe risks.

5.2 What We Tested

Our proxy experiments explored six dimensions of the training recipe:

5.2.1 1. Learning Rate Schedule

The **learning rate** controls how much the model updates its parameters at each training step. Too high and training becomes unstable (loss explodes). Too low and training is too slow (loss barely decreases). The **schedule** controls how the learning rate changes over the course of training.

We tested four schedule types:

- **Cosine decay**: Starts at peak LR, smoothly decreases following a cosine curve to near zero. This is the most common choice.
- **Warmup-Stable-Decay (WSD)**: Ramps up, holds constant for most of training, then linearly decays in the final 20%. Often more robust than cosine.
- **Linear decay**: Simple ramp down from peak to zero.
- **Cosine with warm restarts**: Multiple cosine cycles, periodically resetting the learning rate. This is what our initial (failed) 1B run used.

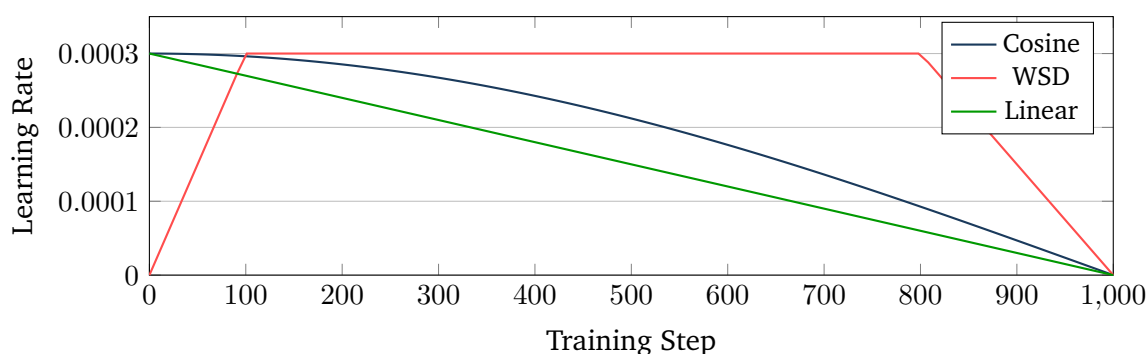


Figure 8: Three learning rate schedules. Cosine (blue) decays smoothly throughout. WSD (red) holds the learning rate steady for most of training, then decays sharply at the end. Linear (green) decays at a constant rate. Our final recipe uses cosine decay.

5.2.2 2. Optimizer Choice

The **optimizer** is the algorithm that uses gradients (how wrong the model was) to update parameters (making the model less wrong). The two main contenders:

- **AdamW** [Loshchilov and Hutter, 2019]: The standard optimizer for training transformers. Maintains running averages of gradients and their squares to adapt the learning rate per-parameter. The “W” means it applies weight decay correctly (a regularization technique that prevents parameters from growing too large).
- **Muon**: A newer optimizer that operates differently on different parameter shapes. Potentially faster convergence, but less tested and harder to tune.

Our proxy experiments started with Muon (which had been used in our earlier 1B attempt) but found it unstable at the learning rates we needed. We switched to AdamW for reliability.

5.2.3 3. Peak Learning Rate

We tested peak learning rates from $1\text{e-}4$ to $8\text{e-}4$. Our final choice: **$3\text{e-}4$** , which is a common default for AdamW with transformer models. Higher rates trained faster initially but sometimes caused instability; lower rates were more stable but slower.

5.2.4 4. Training Duration (Steps)

How long should you train? We tested runs from 200 to 2000 proxy steps (equivalent to different amounts of data). The key finding: validation loss kept improving through our entire proxy range, with no sign of overfitting. This gave us confidence that the full 20,000-step run would not overtrain.

5.2.5 5. Weight Decay

Weight decay is a regularization technique that gently pushes all parameters toward zero, preventing any single parameter from becoming too large. We tested values from 0.01 to 0.1. Our final choice: **0.1**, which is aggressive but standard for large language models.

5.2.6 6. Stochastic Weight Averaging (SWA)

SWA maintains a running average of the model’s parameters over the last portion of training. The averaged model is often slightly better than any single checkpoint because it smooths out the noise of individual training steps. We tested starting SWA at the last 10%, 20%, and 30% of training.

5.3 The Proxy Architecture

Our proxy model is a scaled-down version of the full architecture:

Parameter	Proxy (110M)	Full (3.14B)
Hidden dimension	768	3,072
Layers	12	26
Attention heads	12	24
Training steps	200–2,000	20,000
Time per run	2–5 minutes	22.4 hours
GPU	1 × L40S	8 × H100

Table 9: Proxy vs. full model. The proxy is $\sim 28\times$ smaller and trains $\sim 250\times$ faster, enabling rapid iteration.

The proxy uses the same tokenizer, same data, same optimizer, and same code—just smaller dimensions. This maximizes the chance that findings transfer to the full scale.

5.4 What We Found

Across 32 experiments, here are the key findings:

1. **AdamW beats Muon for reliability.** Muon showed promising early convergence but was sensitive to learning rate and caused training explosions in 3 of 8 runs. AdamW never diverged.
2. **Cosine schedule slightly outperformed WSD.** Both worked well, but cosine was more consistent across different learning rates.
3. **Peak LR of $3e-4$ was optimal.** $5e-4$ was faster but occasionally unstable. $1e-4$ was too slow.
4. **SWA helped modestly.** Starting at the last 20% of training gave small but consistent improvements.
5. **Label smoothing and gradient scaling were unnecessary.** Both added complexity without improving results on bf16 hardware.

Repository Reference

Proxy experiment framework:

train.py	proxy/ Proxy training script
run_agent.py	Autonomous experiment runner
setup_and_run.sh	Environment setup
program.md	Experiment program (what to test and why)

Experiment configs:

exp_000_train.py through exp_032_train.py	proxy/configs/ All 32 configurations
---	---

Results:

results.tsv	proxy/results/ All results in tabular format
muon_arch_results.log	Muon vs. AdamW comparison
tokenizer_training_results.json	Tokenizer variant results

Recipe rationale:

	proxy/rationale/recipe_selection.md
--	-------------------------------------

5.5 Lessons for Your Own Proxy Experiments

1. **Keep the proxy as similar to the full run as possible.** Same code, same data, same tokenizer. Only change the model size and training duration.
2. **Use a consistent evaluation metric.** We used validation BPB (bits per byte) measured on a held-out set. Compare all experiments on the same metric.
3. **Test one variable at a time when possible.** If you change both the optimizer and the learning rate, you will not know which caused the improvement.
4. **Run failing experiments too.** Some of our most useful findings came from understanding *why* an experiment failed (e.g., Muon divergence at high LR).
5. **Budget 10–20 proxy runs minimum.** Fewer than that and you are guessing. More than 50 and you are probably over-optimizing for the proxy scale.

Practical Tip

Proxy experiments are also excellent for testing data pipeline changes. If you suspect your Arabic data has quality issues, run a proxy with and without Arabic and compare. This takes 5 minutes instead of discovering the problem 15 hours into a full run.

Chapter 6: Pretraining: Teaching the Model to Read

Pretraining is where the model learns language. You feed it billions of tokens of raw text, and through the simple task of predicting the next token, it learns grammar, vocabulary, facts, and—crucially for our purposes—cross-lingual structure. This is the most expensive phase of the project and the one where mistakes are most costly.

6.1 What Happens During Pretraining

At a high level, each training step works like this:

1. **Load a batch.** Pull a batch of text sequences from the training data. Each sequence is 2,048 tokens long. With our batch size of 512K tokens, that is 250 sequences per step.
2. **Forward pass.** Feed the batch through the model. For each position in each sequence, the model predicts what the next token should be.
3. **Compute loss.** Compare the model's predictions to the actual next tokens. The **cross-entropy loss** measures how wrong the model was—lower is better.
4. **Backward pass.** Compute **gradients**: for each of the 3.14 billion parameters, calculate how much changing that parameter would reduce the loss. This is done automatically by PyTorch's autograd system.
5. **Update parameters.** The optimizer (AdamW) uses the gradients to adjust each parameter slightly. The learning rate controls how large these adjustments are.
6. **Repeat.** Go back to step 1. Our training runs for 20,000 steps.

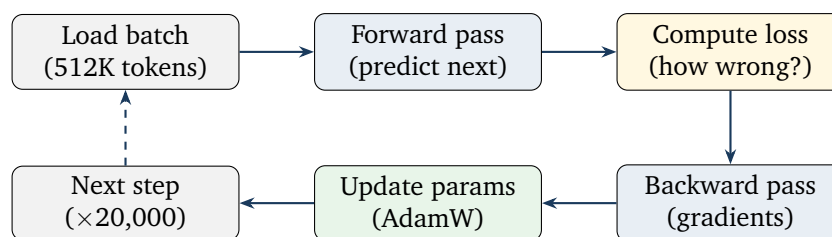


Figure 9: The training loop. Each of 20,000 steps processes 512K tokens. Over the full run, the model sees 4.48B tokens.

6.2 Distributed Training with FSDP

Our model has 3.14 billion parameters. In bfloat16 precision, each parameter takes 2 bytes. So the model itself is ~ 6.3 GB. But during training, we also need:

- **Gradients:** One gradient per parameter (~ 6.3 GB)
- **Optimizer states:** AdamW stores two running averages per parameter (~ 12.6 GB in float32)
- **Activations:** Intermediate values saved for the backward pass (~ 10 – 20 GB depending on batch size)

Total memory: roughly 35–45 GB. A single H100 has 80 GB of memory, so technically it fits, but the batch size would be tiny. To train efficiently, we distribute across **8 GPUs** using a technique called **FSDP (Fully Sharded Data Parallelism)** [Zhao et al., 2023].

6.2.1 How FSDP Works

Imagine you have 8 GPUs, each with 80 GB of memory. Without FSDP, each GPU holds a *complete copy* of the model plus optimizer states, wasting 7/8 of the total memory on redundant copies.

FSDP **shards** (splits) the model across GPUs. Each GPU holds only 1/8 of the parameters, 1/8 of the gradients, and 1/8 of the optimizer states. When a layer needs the full parameters for computation, FSDP temporarily gathers them from all GPUs, computes, and then discards the non-local copies.

Component	Without FSDP (per GPU)	With FSDP (per GPU)
Model parameters	6.3 GB	0.8 GB
Gradients	6.3 GB	0.8 GB
Optimizer states	12.6 GB	1.6 GB
Activations	~ 15 GB	~ 15 GB
Total	~ 40 GB	~ 18 GB

Table 10: Memory usage per GPU with and without FSDP (8 GPUs). FSDP reduces per-GPU memory by more than half, allowing larger batch sizes or larger models.

The trade-off is **communication**: FSDP must send parameter shards between GPUs before and after each layer’s computation. On modern hardware with fast interconnects (like the NVLink in our p5en node), this overhead is small—typically 5–15% of total training time.

6.2.2 Mixed Precision: bfloat16

Training in full 32-bit floating point (float32) would double all the memory numbers above. Instead, we use **bfloat16** (brain floating-point 16), a format that uses half the memory while maintaining the same dynamic range as float32.

The key properties of bfloat16:

- **16 bits** instead of 32 → half the memory, roughly $2\times$ faster computation.
- **Same exponent range as float32** → can represent very large and very small numbers.
- **Less precision in the mantissa** → slightly noisier computations, but this rarely matters in practice.

We use bfloat16 for the forward and backward passes (where speed matters most) and float32 for the optimizer state updates (where numerical precision matters most). This is called **mixed precision** training.

Practical Tip

bfloat16 is only available on relatively modern GPUs (NVIDIA A100, H100, L40S, and newer). Older GPUs (V100, T4) support float16 instead, which has a narrower dynamic range and requires more careful loss scaling. If you are using older hardware, you will need a GradScaler—but on modern hardware, bfloat16 “just works.”

6.3 Training Configuration

Here is our complete training configuration, with explanations:

Parameter	Value	Why
Optimizer	AdamW	Reliable, well-understood. Validated in proxy experiments over Muon.
Peak learning rate	3e-4	Standard for this model size. Validated in proxy.
LR schedule	Cosine decay	Smooth decay, slight edge over WSD in proxy.
Warmup	2,000 steps	Gradual ramp-up prevents early instability. 10% of total steps.
Min LR	3% of peak	Model continues learning slowly at end of training rather than stopping.
Batch size	512K tokens	250 sequences of 2,048 tokens. Balances GPU utilization and gradient quality.
Gradient accumulation	32 steps	Each GPU processes a micro-batch of 8 sequences; accumulate 32 of these for an effective batch of 256.
Weight decay	0.1	Regularization. Standard for LLMs.
Gradient clipping	1.0	Prevents training instability from occasional large gradients.
β_1, β_2	0.9, 0.95	AdamW momentum parameters. 0.95 for β_2 is standard for LLMs (default is 0.999).
Precision	bfloat16	Half the memory, roughly double the speed, no quality loss.

Table 11: Complete pretraining configuration.

6.4 Monitoring Training

A 22-hour training run needs monitoring. Here are the signals we track and what they mean:

6.4.1 Training Loss

The most important metric. Training loss should decrease steadily. If it suddenly spikes up, something is wrong (usually a learning rate issue or data corruption). If it plateaus, the model may have stopped learning (learning rate too low, or all useful patterns already learned).

6.4.2 Validation Loss and BPB

Every 500 steps, we evaluate on a held-out validation set. **BPB (Bits Per Byte)** normalizes the loss across languages with different scripts. A BPB of 3.0 means the model needs 3 bits to encode each byte of text. Lower is better.

Our BPB trajectory:

Step	HE	AR	EN	FA	Note
500	3.87	5.31	5.75	4.79	Early training—all languages high
5,000	2.13	4.30	4.33	3.70	Hebrew learning fastest (anchor effect)
10,000	1.93	4.02	4.09	3.44	Steady improvement, no plateau
15,000	1.82	3.84	3.91	3.26	Decay phase begins
20,000	1.75	3.73	3.83	3.14	Final—40 consecutive new bests

Table 12: BPB evolution during pretraining. Hebrew (the anchor language with 40% data share) learns fastest and achieves the best final BPB. All languages improve continuously with no plateau—a sign of healthy training.

6.4.3 Tokens Per Second (TPS)

Measures training throughput. On our 8×H100 node, we achieved approximately 200K tokens per second. This determines wall-clock time: $4.48\text{B tokens} \div 200\text{K TPS} \approx 22,400 \text{ seconds} \approx 6.2 \text{ hours}$. The actual run took 22.4 hours because evaluation checkpoints, gradient accumulation, and FSDP communication add overhead.

6.4.4 GPU Memory and Utilization

All 8 GPUs should be near 100% utilization during training. If one GPU has lower utilization, FSDP sharding may be unbalanced. If memory usage is near the limit (>75 GB on an 80 GB GPU), reduce batch size.

6.5 Checkpointing: Saving Your Progress

Checkpointing means periodically saving the model’s state to disk (or S3 in our case). This serves two purposes:

1. **Fault tolerance.** Cloud GPU instances can be interrupted (especially spot instances, which we used). Without checkpoints, an interruption means starting from scratch.
2. **Model selection.** The best model is not always the final one. We save checkpoints every 250 steps and track which has the best validation loss.

We save checkpoints to Amazon S3 (cloud storage) rather than local disk, so they survive even if the instance is terminated.

Warning

With FSDP, checkpointing is more complex than simply calling `torch.save()`. You must gather the full model state from all GPUs to rank 0 before saving. Our training script handles this with PyTorch’s `FULL_STATE_DICT` configuration. This is a common source of bugs—test your checkpoint saving and loading *before* starting a long run.

Repository Reference

FSDP training script:	<code>pretrain/scripts/train_multilingual_3b_fsd.py</code>
Training configuration:	<code>pretrain/config/training_config.json</code>
Training logs:	<code>pretrain/logs/</code>
<code>3b_v1_fsd_eval_results.json</code>	Evaluation results at each checkpoint
<code>3b_v1_training.log</code>	Raw training log

6.6 Summary: Pretraining Checklist

1. **Before starting:** Verify your recipe with proxy experiments (Chapter 5). Test checkpoint save/load. Verify data integrity.
2. **Hardware:** Choose the largest node you can afford. FSDP handles multi-GPU automatically.
3. **Configuration:** Use the validated recipe. Do not change hyperparameters from what you tested at proxy scale.
4. **Monitoring:** Watch training loss (should decrease), validation BPB (should decrease), and TPS (should be stable).
5. **Checkpointing:** Save every 250–500 steps to cloud storage. Track the best checkpoint.
6. **Completion:** Keep the best checkpoint (by validation loss) and the final checkpoint. You will use both.

Chapter 7: Supervised Fine-Tuning: Teaching the Model to Work

After pretraining, you have a model that can predict text fluently in four languages. But it cannot follow instructions, classify sentiment, or perform any specific task. It is like a well-read person who has never been asked a question. **Supervised fine-tuning (SFT)** bridges this gap.

7.1 What SFT Does

SFT takes the pretrained model and trains it further on **labeled examples** of the behavior you want. The key differences from pretraining:

	Pretraining	SFT
Data	Raw text (unlabeled)	Input-output pairs (labeled)
Objective	Predict next token	Predict correct output
Scale	Billions of tokens	Tens of thousands of examples
Duration	Hours–days	Minutes–hours
Learning rate	High ($3e-4$)	Low ($2e-5$)
Goal	Learn language	Learn behavior

Table 13: Pretraining vs. SFT. SFT is faster, cheaper, and uses much less data, but it requires labeled examples.

Key Idea

SFT is where your model goes from “knows language” to “does useful things.” The pre-trained model has encoded vast linguistic knowledge, but SFT teaches it *when* and *how* to use that knowledge. Think of pretraining as education and SFT as job training.

7.2 Our SFT Data

We created two types of training examples:

7.2.1 Sentiment Classification (18,980 samples)

Each example is a text paired with a sentiment label (positive or negative):

Example: Hebrew sentiment

```
1 <|user|> Classify the sentiment of this text as positive or  
   negative:  
2 "The restaurant was amazing, best food I've had in years"  
3 <|assistant|> positive
```

We use native-language datasets for each language:

- **Hebrew:** HebEMO (Hebrew emotional analysis, Israeli tweets)
- **Arabic:** HARD (Hotel Arabic Reviews Dataset)
- **Persian:** DeepSentiPers (Persian product reviews)
- **English:** SST-2 (Stanford Sentiment Treebank)

7.2.2 Translation Pairs (18,000 samples)

Each example is a sentence in one language and its translation in another:

Example: Translation pair

```
1 <|user|> Translate from English to Hebrew:  
2 "The weather is beautiful today."  
3 <|assistant|> hamezeg avir yafe hayom.
```

Translation data comes from OPUS-100, covering all six language-pair directions that include English (EN↔HE, EN↔AR, EN↔FA). Each direction contributes 3,000 pairs.

7.3 The Importance of Data Quality

If Chapter 10 is the most important chapter for practitioners, this subsection is the preview. We discovered—the hard way—that SFT data quality is even more critical than pretraining data quality. Small bugs in labeling or sampling can produce models that appear to work but are actually broken.

The three rules we enforce after painful debugging:

1. **Verify label alignment.** Different datasets encode labels differently. One dataset's 0 might mean "positive" while another's 0 means "negative." Always check.
2. **Shuffle before splitting.** If the original dataset is sorted by label (all positives first, then all negatives), a naive train/test split will give you a training set with only one class.
3. **Zero data leakage.** Never include evaluation examples in the training set, even accidentally. This sounds obvious but can happen through careless file handling.

All three of these rules were violated in our earlier SFT iterations, producing results that looked excellent but were meaningless. See Chapter 10 for the full story.

7.4 SFT Training Configuration

Parameter	Value	Why
Optimizer	AdamW	Same as pretraining, proven reliable.
Learning rate	2e-5	15× lower than pretraining. SFT must make small adjustments without destroying pretrained knowledge.
Schedule	Cosine decay	Consistent with pretraining.
Steps	8,000	Enough to see all 37K examples ~5 times.
Sequence length	384	SFT examples are short (a sentence + label). Using 384 instead of 2,048 saves memory and speeds training 5×.
Batch size	8 sequences	Small batch is fine for fine-tuning—the model is already well-initialized.
Precision	bfloat16	Same as pretraining.

Table 14: SFT training configuration.

Warning

The SFT learning rate (2e-5) is much lower than the pretraining learning rate (3e-4). This is critical. If you use a high learning rate during SFT, you will **catastrophically forget** the knowledge learned during pretraining. The model will become good at the SFT tasks but lose its general language abilities. This phenomenon is called **catastrophic forgetting** and is one of the most common SFT mistakes.

7.5 The English-Only SFT Control Experiment

To test whether English-only fine-tuning transfers to other languages, we also train a separate model using **only the English subset** of our SFT data (4,745 examples). Everything else is identical: same pretrained base model, same optimizer, same learning rate. The only difference is the language of the training data.

This control is the most important experiment in the paper. If the English-only model also improves on Hebrew, Arabic, and Persian, that would mean cross-lingual transfer is happening. If it does not improve, that proves multilingual SFT data is necessary.

Spoiler: English-only SFT did *not* transfer. The results are in Chapter 9.

7.6 SFT Output

SFT produces a new set of model weights (6.3 GB), saved alongside the base pretrained model. The SFT model has the same architecture as the base model—only the parameter values differ.

Repository Reference

V4 SFT scripts (with all data fixes):

prepare_v4_data.py

run_v4.py

sft/scripts/v4/
Data preparation with all corrections

Training + evaluation script

SFT configurations:

D-baseline.json

E-1K.json, E-3K.json, E-5K.json

F-no-arfa.json, F-only-arfa.json

sft/config/

Full multilingual SFT config

SFT scaling experiments

Language ablation configs

Earlier SFT scripts (pre-fix, for reference):

train_sft_3b.py

prepare_sft_data.py, prepare_sft_data_v2.py

sft/scripts/

SFT training script (v1/v2)

Earlier data prep

Training logs:

D_training.log through F-only-arfa_training.log

sft/logs/

All SFT run logs

7.7 Summary: SFT in Practice

1. **Start with clean data.** Verify labels, shuffle, enforce no leakage.
2. **Use a low learning rate.** 10–20× lower than pretraining.
3. **Keep it short.** A few thousand steps is usually sufficient.
4. **Include all target languages.** At 3B scale, English-only SFT does not transfer.
5. **Run control experiments.** An English-only control lets you measure exactly how much multilingual data matters.
6. **Small amounts of data suffice.** Our SFT used only 37K total examples. Quality matters more than quantity.

Practical Tip

SFT is cheap. On a single A10G GPU (about \$1/hour), our full SFT runs took about an hour each. This means you can iterate rapidly—fix a data bug, retrain, re-evaluate, all in an afternoon. Pretraining does not afford this luxury, which is why getting it right (through proxy experiments) matters so much.

Chapter 8: Evaluation: Measuring What Your Model Learned

You have trained a model. It consumed 4.48 billion tokens during pretraining and 37,000 labeled examples during SFT. But how do you know it actually *works*? Evaluation is the process of measuring model capabilities in a controlled, reproducible way. Done poorly, evaluation will tell you your model is brilliant when it is broken (as we learned firsthand). Done well, it gives you honest, actionable information.

8.1 Why Evaluation Is Harder Than It Looks

Evaluating a language model is not like evaluating a calculator. A calculator either gives the right answer or it does not. A language model operates in a space of probabilities and generated

text, and there are many ways to ask it a question, many ways to interpret its answer, and many ways to accidentally fool yourself.

Here are the main pitfalls:

1. **Evaluation data contamination.** If your evaluation examples accidentally appeared in training data, your results are meaningless—the model is reciting memorized answers, not demonstrating understanding.
2. **Label errors.** If your “ground truth” labels are wrong, the model can be penalized for correct answers and rewarded for wrong ones.
3. **Imbalanced evaluation sets.** If 90% of your sentiment examples are positive, a model that always says “positive” gets 90% accuracy. You need balanced sets (50/50) so that chance performance is clearly 50%.
4. **Parsing fragility.** If you ask the model to generate “positive” or “negative” and it generates “Positive” or “pos” or “The sentiment is positive,” do you count that as correct?

We hit all four of these problems during development. The evaluation methodology described below is the result of fixing them.

8.2 Our Evaluation Methods

We use three complementary evaluation approaches, each measuring something different:

8.2.1 Method 1: Logprob Classification

What it measures: Does the model assign higher probability to the correct label?

How it works: Given a text and a classification prompt, we compute the **log-probability** that the model assigns to the token “positive” versus the token “negative” as the next token. Whichever has the higher probability is the model’s prediction.

Classify as positive or negative: “Great movie!”

$P(\text{“positive”} \mid \text{prompt}) = 0.82$

← **Winner**

$P(\text{“negative”} \mid \text{prompt}) = 0.11$

Figure 10: Logprob classification. Instead of generating text, we directly compare the probability the model assigns to each label. This eliminates parsing ambiguity entirely.

Why this is better than generation: There is no text to parse. The model either assigns higher probability to the correct label or it does not. No formatting ambiguity, no partial matches, no prompt sensitivity.

Limitations: It only works for classification tasks where you can define a small set of possible labels. You cannot use this method for open-ended generation tasks.

Our setup: 200 examples per language, balanced 50/50 positive/negative. Chance performance = 50%.

8.2.2 Method 2: Generative Classification

What it measures: Can the model follow an instruction and produce a correctly formatted answer?

How it works: Give the model a prompt like “Classify this text as positive or negative” and let it generate a free-form response. Then parse the response for sentiment labels.

Why include this: This is closer to how real users interact with models. A model that “knows” the answer (high logprob) but cannot *express* it (fails generative) has a practical limitation.

Limitations: Fragile. The model might understand the task perfectly but produce output like “I think the sentiment is mostly positive with some caveats” which fails a strict parser. We mitigate this by accepting multiple formats (“positive,” “Positive,” “pos,” etc.).

Our setup: 50 examples per language, balanced 50/50. Chance performance = 0% (the model must produce a parseable label; random text will not match).

8.2.3 Method 3: Cross-Lingual Retrieval

What it measures: Does the model understand that the same meaning in different languages should have similar internal representations?

How it works:

1. Take a sentence in language A (e.g., English).
2. Take 10 sentences in language B (e.g., Hebrew)—one is the correct translation, nine are random distractors.
3. Feed all 11 sentences through the model and extract their **embeddings** (internal vector representations) from the last hidden layer.
4. Measure which of the 10 Hebrew sentences has the most similar embedding to the English sentence (using cosine similarity).
5. If the correct translation is most similar, count it as a hit.

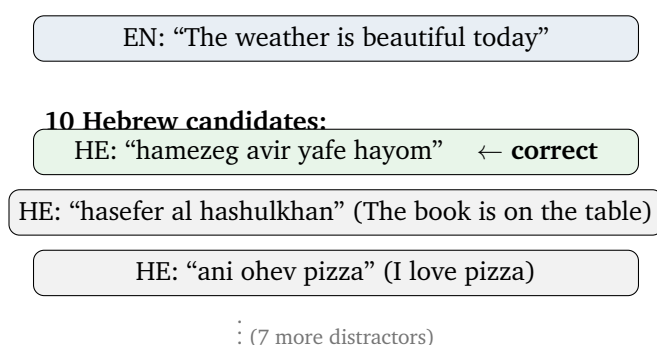


Figure 11: Cross-lingual retrieval. The model must identify the correct translation from 10 candidates using only its internal representations. Chance = 10%.

Why this matters: Retrieval accuracy measures the quality of the model’s *internal representations*, not its generation ability. High retrieval accuracy means the model has learned that the same concept in different languages should “look similar” internally. This is the foundation of cross-lingual transfer.

Our setup: 10-way retrieval for all language pairs. Chance = 10%.

8.2.4 Additional Evaluation: Belebele Reading Comprehension

Belebele [Bandarkar et al., 2023] is a standardized reading comprehension benchmark available in 122 languages. Each question gives a passage and four answer choices. We include it to test whether our model can perform abstract reasoning, not just classification.

Spoiler: at 3B scale, it cannot. All results are near the 25% chance level. This is expected and honest—it tells us where our model’s capabilities end.

8.3 Evaluation Set Construction

The quality of your evaluation sets is as important as the quality of your training data. Here are the rules we follow:

1. **Balance all classes.** Every evaluation set has exactly 50% positive and 50% negative examples (for binary classification). This makes chance performance exactly 50%, which is a clean baseline.
2. **Sufficient size.** We use 200 examples per language for logprob (statistical power for detecting >5pp differences) and 50 for generative (slower to run).
3. **Native-language sources.** Each language’s evaluation data comes from a *native* dataset in that language—not translations from English. This ensures we are testing real-world text, not translation artifacts.
4. **Strict separation.** Evaluation examples are held out before any training begins. They are never seen during pretraining or SFT.
5. **Shuffled sampling.** We shuffle the source dataset before selecting evaluation examples, ensuring a random distribution of topics, authors, and styles.

Warning

We cannot stress this enough: **verify your evaluation sets manually**. In our early experiments, we discovered that 31.4% of our Hebrew evaluation labels were wrong (due to a label inversion bug), and our Arabic evaluation set contained only negative examples (due to a sorting artifact). Both bugs produced results that looked plausible—they just happened to be completely wrong. See Chapter 10 for details.

8.4 Interpreting Results: What Counts as “Good”?

When you see that your model gets 84.5% on Hebrew sentiment, is that good? Context matters:

Benchmark	What counts as good
Logprob sentiment (chance=50%)	70%+ is meaningful; 85%+ is strong
Generative sentiment (chance=0%)	50%+ shows task understanding; 80%+ is strong
Cross-lingual retrieval (chance=10%)	50%+ is meaningful; 80%+ is strong
Belebele (chance=25%)	40%+ at 3B would be notable

Table 15: Rough interpretation guide. These thresholds are specific to our model size (3B) and evaluation setup.

Comparing against larger models is useful for context but not the primary goal. Our research questions are about *relative* performance: does multilingual SFT beat English-only SFT? Does the tokenizer affect efficiency? These comparisons use the same model and same evaluation, making them internally valid regardless of absolute performance.

Repository Reference

Evaluation scripts:	eval/scripts/
exp_a_fix_classification.py	Sentiment classification (logprob + generative)
exp_b_crosslingual.py	Cross-lingual BPB evaluation
eval_embeddings.py	Cross-lingual retrieval
eval_belebele.py	Belebele reading comprehension
exp_c_tokenizer_ablation.py	Tokenizer efficiency comparison
tokenizer_analysis.py	Detailed tokenizer metrics
Evaluation results:	eval/results/
v4_results.json	Final v4 results (post-data-fix)
exp_a_fix_results.json	Experiment A sentiment results
exp_ab_results.json	Combined A+B results
exp_c_tokenizer_ablation.json	Tokenizer comparison data
belebele_3b_results.json	Belebele benchmark results
Evaluation prompts:	eval/prompts/prompts.md

8.5 Summary: Evaluation Principles

1. **Use multiple evaluation methods.** No single metric tells the whole story. Logprob measures knowledge; generative measures usability; retrieval measures representations.
2. **Balance your evaluation sets.** Unbalanced sets hide model weaknesses.
3. **Use native-language data.** Translated evaluation data introduces artifacts.
4. **Verify labels manually.** Spot-check at least 50 examples per language.
5. **Report chance baselines.** Always state what random performance would be, so readers can judge whether your results are meaningful.
6. **Include negative results.** Report tasks where your model fails (like our Belebele results). This is honest and informative.

Practical Tip

Build your evaluation pipeline *before* you start training, and run it on the base model first. The base model results establish your baseline and verify that the evaluation code works correctly. If your base model gets 95% accuracy on a balanced binary task, something is wrong with your evaluation, not right with your model.

Chapter 9: Results: What SemiticGPT Can Do

This chapter presents our experimental results. We organize them around three questions, each addressed by a dedicated experiment.

9.1 Experiment A: Does Multilingual SFT Work?

The most basic question: after pretraining and multilingual fine-tuning, can the model actually classify sentiment across all four languages?

Stage	HE	AR	FA	EN
<i>Logprob classification (N=200, chance=50%)</i>				
Base (pretrained only)	53.0	45.0	60.5	51.5
EN-SFT (English only)	51.5	46.5	58.5	52.0
Multi-SFT	84.5	60.5	78.5	73.0
<i>Generative classification (N=50, chance=0%)</i>				
Base	0	0	0	0
EN-SFT	0	0	0	2
Multi-SFT	82	64	74	64

Table 16: Sentiment classification accuracy (%) across three model stages. The three-stage comparison separates what the model learns from pretraining alone, from English-only task training, and from multilingual task training.

How to read this table:

- **Base model** (row 1): Near chance (50%) for logprob, zero for generative. The pre-trained model has no task-following ability. It “knows” these languages but cannot use that knowledge for classification.
- **EN-SFT** (row 2): Almost no improvement. Training on English-only examples does not help the model classify sentiment in *any* language, including English itself. This is the key negative result.
- **Multi-SFT** (row 3): Large gains everywhere. Hebrew improves by 31.5 percentage points (logprob), Arabic by 15.5pp, Persian by 18pp, English by 21.5pp.

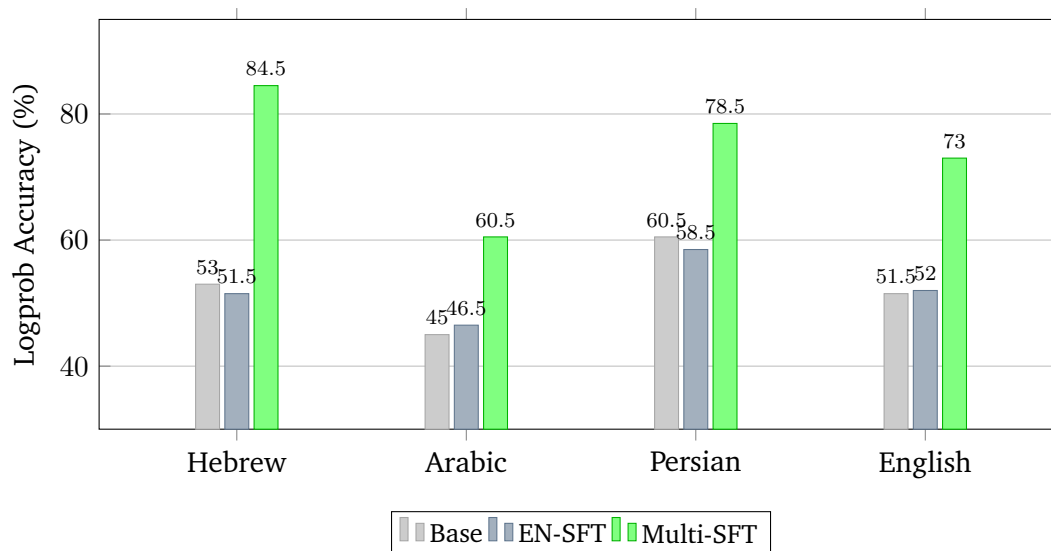


Figure 12: The visual story: the gray bars (base) and blue bars (EN-SFT) are nearly identical, meaning English-only fine-tuning does almost nothing. The green bars (Multi-SFT) show substantial improvement across all languages.

Key Idea

The central finding: at 3B scale with 4.48B pretraining tokens, **English-only SFT is insufficient for cross-lingual task transfer**. The model needs explicit examples in each target language to perform downstream tasks. Multilingual pretraining creates shared representations, but those representations cannot be activated for tasks without multilingual fine-tuning.

9.2 Experiment B: Cross-Lingual Transfer Analysis

Experiment A showed that multilingual SFT works. Experiment B asks *why* English-only SFT does not, and what the pretrained model *has* learned about cross-lingual structure.

9.2.1 The Pretrained Model Has Cross-Lingual Representations

Despite the EN-SFT failure, the base model develops strong cross-lingual alignment purely from pretraining:

Direction	Base	Multi-SFT
EN→HE	90%	90%
HE→EN	90%	80%
AR→EN	70%	50%
AR→HE	70%	70%
EN→AR	50%	60%
HE→AR	40%	60%
FA→HE	50%	30%
HE→FA	40%	30%
EN→FA	20%	20%
AR→FA	10%	10%
FA→EN	10%	10%
FA→AR	10%	10%
Average	45.8%	43.3%

Table 17: Cross-lingual retrieval accuracy (10-way, chance=10%). English↔Hebrew retrieval reaches 90%—9× chance—purely from pretraining.

This is a striking result: the model places English and Hebrew sentences with the same meaning very close together in its internal representation space, even though it was never explicitly trained to do so. Cross-lingual alignment *emerged* from the simple task of predicting the next token across multilingual data.

9.2.2 A Language Family Hierarchy

The retrieval results reveal a clear hierarchy of cross-lingual connectivity:

1. **English ↔ Hebrew: 90%** — Strongest pair. Hebrew is the anchor language (40% of data) and English is the bridge language. Both are well-represented.
2. **Arabic ↔ English/Hebrew: 40–70%** — Moderate connectivity. Arabic shares Semitic roots with Hebrew, which helps.
3. **Persian ↔ everything: 10–50%** — Weak connectivity. Despite sharing Arabic script, Persian is typologically distant (Indo-European, not Semitic).

This pattern is consistent with **linguistic family membership being a stronger predictor of cross-lingual transfer than script similarity**. Arabic script does not automatically create alignment between Arabic and Persian; shared Semitic grammar creates alignment between Arabic and Hebrew.

9.2.3 The Representation–Task Gap

Here is the puzzle: the base model achieves 90% EN↔HE retrieval (strong representations) but only 53% Hebrew sentiment accuracy (near chance). How can it “know” these languages are related but be unable to use that knowledge?

The answer is that **representation alignment** and **task transfer** are different capabilities:

- **Representation alignment** means the model’s internal vectors for similar concepts are close together across languages. This emerges naturally from multilingual pretraining.
- **Task transfer** means the model can apply a learned behavior (like sentiment classification) across languages. This requires explicit instruction.

An analogy: imagine you have a detailed multilingual map (representation alignment). You can see that Tel Aviv and Paris are both cities. But knowing they are both cities does not tell you how to navigate from one to the other—you need explicit directions (task transfer via SFT).

9.2.4 The Retrieval–SFT Trade-off

An interesting secondary finding: after multilingual SFT, average retrieval accuracy drops slightly (45.8% → 43.3%). Fine-tuning makes the model better at classification but slightly worse as a general-purpose cross-lingual encoder. This is a common phenomenon: optimizing for one capability can come at a small cost to another. The model’s internal representations become more task-focused and slightly less general-purpose.

9.3 Experiment C: Tokenizer Efficiency

Experiment C is purely about the tokenizer—no model training required. We compare our custom tokenizer against Llama-2 (English-centric) and HebrewGPT (Hebrew-centric), all with 32K vocabularies.

The detailed results were presented in Chapter 3. The headline numbers:

- **Hebrew:** 66% fewer tokens than Llama-2 (1.34 vs 3.91 tokens/word)
- **Arabic:** 49% fewer tokens (2.22 vs 4.36)
- **Persian:** 69% fewer tokens (1.52 vs 4.88)
- **English:** 2% *more* tokens (1.54 vs 1.51)—negligible penalty

In practical terms: for every Hebrew document, our model processes $3\times$ more text in the same context window compared to using Llama-2’s tokenizer. For Persian, it is $3.2\times$ more. This is a massive efficiency gain that costs almost nothing on English.

9.4 Belebele: Where the Model Fails

Belebele reading comprehension results are near chance (25%) for all stages and all languages:

Stage	HE	AR	FA	EN
Random	25.0	25.0	25.0	25.0
Base	26.0	22.0	19.0	19.0
Multi-SFT	28.0	24.0	25.0	23.0

Table 18: Bebelebe reading comprehension (% , 4-way multiple choice, $N=100$). All near chance (25%).

This is expected: Bebelebe requires reading a passage, understanding a question, and selecting the correct answer from four choices. This level of abstract reasoning typically requires models of 7B+ parameters with much larger pretraining corpora. We include these results because **reporting where your model fails is as important as reporting where it succeeds**. It tells practitioners what tasks are out of reach at this scale.

9.5 Summary of Results

Finding	Significance
Multilingual SFT required	English-only fine-tuning does not transfer to other languages at 3B scale. Each language needs its own task examples.
Cross-lingual representations emerge	The base model learns that related languages belong in a shared semantic space, purely from next-token prediction.
Representation $\neq$ task transfer	Good cross-lingual embeddings do not automatically enable cross-lingual task performance.
Anchor language benefits	Hebrew (40% data share) achieves the strongest results across all experiments.
Family > script	Persian shares Arabic script but shows weak transfer. Hebrew–Arabic transfer is stronger, consistent with shared Semitic grammar.
Tokenizer efficiency is critical	49–69% fewer tokens for target languages with minimal English penalty.
Scale limitations are real	3B is insufficient for reading comprehension (Belebele near chance).

Table 19: Summary of experimental findings and their practical implications.

Chapter 10: What Went Wrong: Data Bugs, Failed Runs, and Hard Lessons

This may be the most valuable chapter in the guide. Every published paper shows polished results; very few show the failures that preceded them. Our path to working results included inverted labels, contaminated evaluation, sorting artifacts, crashed training runs, and wasted compute. Each mistake taught us something useful.

10.1 Lesson 1: The Hebrew Label Inversion

What happened: Our Hebrew sentiment dataset (mteb/hebrew_sentiment_analysis) encodes labels as integers: 0 = positive, 1 = negative, 2 = off-topic. Our data preparation script assumed the standard convention: 0 = negative, 1 = positive. Every Hebrew label was flipped.

How we discovered it: We achieved 80% accuracy on Hebrew sentiment, which seemed excellent. But when we manually inspected the predictions, the model was calling clearly negative tweets “positive” and vice versa. A keyword-based audit (checking for positive words in “negative” predictions) confirmed that 31.4% of labels were inconsistent.

The real damage: The 80% accuracy was measuring how well the model learned the *wrong* mapping. It was not a sign of capability—it was a sign that the model was faithfully learning incorrect labels.

The fix: Read the dataset’s documentation. Check the `ClassLabel` object: `names=['pos', 'neg', 'off-topic']`, meaning index 0 = positive, index 1 = negative. Our script now explicitly maps from the dataset’s schema rather than assuming a convention.

Warning

Different datasets encode the same concept differently. SST-2 uses 0=negative, 1=positive. Hebrew Sentiment uses 0=positive, 1=negative. There is no universal standard. **Always read the dataset card** and verify with manual inspection.

10.2 Lesson 2: The Arabic Sorting Artifact

What happened: Our Arabic sentiment dataset (`arbml/Sentiment_Analysis_Tweets`) was sorted by label in the source data: all 29,100 negative tweets first, then all 38,970 positive tweets. Our script loaded the first 10,000 examples for training and the next 500 for evaluation.

Result: both training and evaluation sets contained **only negative examples**.

How we discovered it: The model achieved 100% accuracy on Arabic sentiment. This was suspicious—no model gets 100% on a real task. A simple `value_counts()` on the labels confirmed: zero positive examples in either set.

The real damage: 100% accuracy on a test with only one class is trivially achieved by always predicting that class. The metric was meaningless.

The fix: Always shuffle before splitting. Our data preparation script now enforces: (1) load the full dataset, (2) shuffle with a fixed random seed, (3) *then* split into train/eval. Additionally, we verify that both classes are present in both splits.

Practical Tip

Add a simple assertion to your data pipeline:

```
1 assert len(set(eval_labels)) >= 2, \
2     f"Eval set has only {set(eval_labels)} classes!"
3 assert abs(eval_labels.mean() - 0.5) < 0.1, \
4     f"Eval set imbalanced: {eval_labels.mean():.1%} positive"
```

These two lines would have caught our Arabic bug immediately.

10.3 Lesson 3: Data Leakage

What happened: Our v3 training script accidentally included the evaluation files in the training data. The model was tested on examples it had already memorized.

How we discovered it: When we ran the same evaluation on a freshly prepared dataset (v4, with strict separation), results dropped significantly. The v3 “good results” were partially memorization.

The fix: Strict file-level separation. Evaluation data is prepared in a separate script, saved to separate files, and never loaded by the training script. We verify separation by checking that

no example in the evaluation set appears in the training set (using exact string matching).

Key Idea

Data leakage is insidious because **it produces results that look correct**. The model genuinely gets the right answer—it just does so by remembering, not understanding. The only reliable defense is strict process: separate data pipelines for training and evaluation, with automated verification.

10.4 Lesson 4: Translation Data Quality

What happened: Our v3 translation training data was supposed to contain thousands of real sentence pairs from OPUS-100. Instead, the download URLs had changed (404 errors), and the fallback code silently used 50 hardcoded example pairs. The model’s translation “training” consisted of memorizing 50 sentences.

How we discovered it: By checking the actual file sizes. The translation training file was 4 KB instead of the expected 400 KB.

The fix: For v4, we download directly from HuggingFace’s datasets library (Helsinki-NLP/opus-100), which provides stable access. We also added size checks: if a downloaded dataset has fewer than 1,000 examples, the script raises an error rather than silently proceeding.

10.5 Lesson 5: The Failed 1B Training Run

What happened: Before the successful 3B run, we attempted a 1B parameter model. The training used a cosine schedule with 3 warm restarts over 15,000 steps. The model peaked at step 2,000 (about 1B tokens seen), then validation loss *increased* continuously through step 4,500, at which point we terminated.

Root cause: The cosine schedule with warm restarts decayed the learning rate too aggressively. By step 4,000, the learning rate had dropped to near zero, with 73% of training steps remaining at an ineffective learning rate. The warm restarts briefly increased LR but not enough to recover.

What we learned: This failure directly motivated the proxy experiments (Chapter 5). We tested the fix at 110M scale first, confirmed that a simple cosine schedule (no restarts) with AdamW worked correctly, and only then committed to the 3B run. The 3B run completed successfully with monotonically decreasing loss—40 consecutive new bests, zero plateau.

10.6 Lesson 6: Optimizer Instability

What happened: Our proxy experiments initially used the Muon optimizer, which had shown promising results in other contexts. At learning rates above 0.02, Muon caused training loss to explode (validation BPB jumped from ~15 to ~700). Three of eight Muon experiments diverged.

What we learned: Newer, more exotic optimizers may offer advantages but also carry risks. AdamW is “boring” but reliable—it never diverged in any of our 24 AdamW experiments. For a project where you have one shot at a multi-day training run, reliability matters more than marginal convergence speed.

10.7 Lesson 7: Architecture Mismatch

What happened: Our v2 SFT evaluation loaded the model using a simplified transformer class instead of the actual GPT architecture used during pretraining. The simplified class lacked fused QKV projections and RoPE position encoding. The model weights loaded without errors (the

parameter names happened to match) but the computation was wrong.

How we discovered it: v2 results were mediocre despite “identical” training. A code review revealed the architecture class mismatch.

The fix: Use exactly the same model class for training, SFT, and evaluation. Define the architecture once and import it everywhere. Never copy-paste model code between scripts.

10.8 Lesson 8: Instance Type Mistakes

What happened: We initially launched the 3B training on a g6e.48xlarge (8×L40S) instead of the planned p5en (8×H100). The L40S is 3.5× slower for this workload. We discovered this 15 hours in, having already spent ~\$200 on a run that would have taken 200+ hours total.

The fix: We launched the correct p5en instance, verified the training speed (200K+ TPS vs 14K TPS), confirmed identical loss curves, and terminated the slow instance after saving checkpoints.

What we learned: Always verify your instance type and training speed in the first 30 minutes. A 10× mismatch in throughput means 10× the cost for the same result.

10.9 The Meta-Lesson

Key Idea

Every mistake in this chapter has the same structure:

1. Something *appeared* to work.
2. We did not verify the assumption.
3. Later discovery invalidated the results.

The defense is systematic verification at every stage: check your labels, check your data splits, check your model architecture, check your instance type, check your file sizes. The 10 minutes you spend verifying will save you days of wasted training.

10.10 Verification Checklist

Here is the checklist we now run before every training job:

- ☐ **Labels:** Manually inspect 20+ examples per language. Do the labels match the text?
- ☐ **Balance:** Run `value_counts()` on labels for both train and eval sets.
- ☐ **Separation:** Verify zero overlap between train and eval examples.
- ☐ **Data size:** Check file sizes. Are they in the expected range?
- ☐ **Architecture:** Verify the same model class is used for training and evaluation.
- ☐ **Baseline:** Run evaluation on the base model. Are results near chance?
- ☐ **Instance:** Verify GPU type and training speed in the first 30 minutes.
- ☐ **Checkpointing:** Verify that checkpoint saving and loading works before starting a long run.

Chapter 11: Practitioner’s Checklist: Building Your Own Multilingual Model

This final chapter distills the entire guide into a step-by-step recipe. If you are building a multilingual language model for your own language cluster, this is your roadmap. Each step references the relevant chapter for deeper explanation and the relevant repository artifact for code.

11.1 Phase 1: Define Your Scope (1–2 days)

Before writing any code, answer these questions:

Question	Our Answer	Chapter
Which languages?	Hebrew, Arabic, Persian, English	1
Why these languages?	Semitic family cluster + Persian as control	1
What tasks?	Sentiment classification, cross-lingual transfer	8
What model size?	3B (fits one GPU for inference, trainable on one node)	2
What hardware?	8×H100 (p5en.48xlarge) for pretraining; 1×A10G for SFT	6
What is the anchor language?	Hebrew (40% of pretraining data)	4

Table 20: *Scoping questions. Answer these before starting.*

Practical Tip

Choose your languages based on a **linguistic hypothesis**, not just convenience. We chose Hebrew + Arabic (Semitic siblings) + Persian (same script, different family) + English (bridge) specifically to test whether linguistic relatedness predicts transfer better than script similarity.

11.2 Phase 2: Build the Tokenizer (1–2 days)

1. **Collect balanced text samples.** Equal proportion from each target language. 100MB–1GB per language is sufficient for tokenizer training.
2. **Train SentencePiece BPE.** Use 32K vocabulary for 4–6 languages (64K if more). Enable byte fallback and NFKC normalization.
3. **Add special tokens.** Define `<s>`, `</s>`, `<pad>`, and any instruction tokens (`<|user|>`, `<|assistant|>`) before training.
4. **Verify fertility.** Compute tokens-per-word on held-out text for each language. Target: ≤ 2.0 for all languages. Check that English penalty is $< 5\%$.
5. **Inspect vocabulary composition.** Check the script distribution in the vocabulary. It should roughly reflect your language mix.

Repository Reference

Script: tokenizer/scripts/train_tokenizer.py
tokenizer/config/tokenizer_config.json

Config:
Output: tokenizer/artifacts/

11.3 Phase 3: Collect and Prepare Data (1–2 weeks)

This is the most time-consuming phase. Do not rush it.

1. **Identify sources.** For each language: Wikipedia, OSCAR/Common Crawl, news corpora, government documents, domain-specific text.
2. **Download and deduplicate.** Use MinHash deduplication to remove near-duplicates.
3. **Filter for quality.** Run language identification to remove mislabeled documents. Apply perplexity-based filtering using a small reference model.
4. **Tokenize.** Convert clean text to token IDs using your custom tokenizer. Store as uint16 binary files.
5. **Mix to target proportions.** Combine language-specific files according to your target distribution (e.g., 40/20/20/20).
6. **Create manifests.** Record every source, its license, size, and processing steps. You will need this for reproducibility and debugging.
7. **Verify.** Spot-check 100+ decoded examples per language. Do they look like clean, natural text in the expected language?

Repository Reference

Collection scripts: data/scripts/

Manifests: data/manifests/

11.4 Phase 4: Run Proxy Experiments (2–3 days)

1. **Define a proxy model.** Scale down your architecture by $\sim 25\text{--}30\times$. Our 3B model scaled down to 110M for proxies.
2. **Write the experiment program.** List what you want to test, in priority order: LR schedule, optimizer, peak LR, training duration, weight decay, regularization.
3. **Run 20–40 experiments.** Each should take 2–10 minutes. Use the same data and tokenizer as the full run.
4. **Select the winner.** Choose the configuration with the best validation BPB, breaking ties by simplicity and stability.
5. **Document everything.** Record all configs, results, and rationale. You will reference this when debugging the full run.

Repository Reference

Framework: proxy/train.py, proxy/run_agent.py
proxy/configs/exp_000-032_train.py

Configs:
Results: proxy/results/results.tsv

11.5 Phase 5: Pretrain (1–3 days)

1. **Launch your instance.** Verify GPU type, count, and interconnect speed. Check in the first 30 minutes.
2. **Configure FSDP.** Set sharding strategy, mixed precision (bf16 compute, fp32 reduce), and activation checkpointing.

3. **Apply proxy recipe.** Use exactly the hyperparameters validated at proxy scale. Do not tweak.
4. **Enable checkpointing.** Save to cloud storage (S3, GCS) every 250–500 steps. Track the best checkpoint by validation loss.
5. **Monitor.** Watch training loss (should decrease), validation BPB (should decrease), TPS (should be stable). Set up alerts for loss spikes.
6. **Let it finish.** Resist the temptation to stop early unless something is clearly wrong. Our model improved for all 20,000 steps—40 consecutive new bests.

Repository Reference

Training script: `pretrain/scripts/train_multilingual_3b_fsdp.py` **Config:** `pretrain/config/training_config.json`
Logs: `pretrain/logs/`

11.6 Phase 6: Prepare SFT Data (2–3 days)

This phase requires the most care per line of code.

1. **Select native-language datasets.** For each language and task, find a dataset created in that language (not translated from English).
2. **Verify labels.** Read the dataset card. Check the `ClassLabel` mapping. Manually inspect 20+ examples.
3. **Shuffle before splitting.** Always. Then verify both classes appear in both train and eval.
4. **Enforce balance.** Evaluation sets must be exactly 50/50 (for binary tasks).
5. **Strict separation.** Training and evaluation data must come from separate, non-overlapping partitions.
6. **Add translation data.** If you want cross-lingual capabilities, include parallel sentence pairs from OPUS-100 or similar.
7. **Format for instruction tuning.** Wrap examples in `<|user|>...<|assistant|>` format.
8. **Run the verification checklist** from Chapter 10 before proceeding.

Repository Reference

V4 data preparation (with all fixes): `sft/scripts/v4/prepare_v4_data.py` **SFT configs:** `sft/config/`

11.7 Phase 7: Supervised Fine-Tuning (1 day)

1. **Start from the best pretrained checkpoint.**
2. **Use a low learning rate.** 10–20× lower than pretraining (e.g., 2e-5 if pretraining used 3e-4).
3. **Train for a few thousand steps.** Monitor validation loss; stop if it starts increasing (overfitting).

4. **Run a control experiment.** Train an English-only SFT variant to measure whether cross-lingual transfer happens at your scale.
5. **Save the best model.** By validation loss, not the final step.

Repository Reference

Training script: `sft/scripts/v4/run_v4.py`Logs: `sft/logs/`

11.8 Phase 8: Evaluate (1–2 days)

1. **Run all evaluation methods.** Logprob classification, generative classification, cross-lingual retrieval, and any task-specific benchmarks.
2. **Evaluate all model stages.** Base, EN-SFT control, and multilingual SFT. The three-stage comparison tells the full story.
3. **Include negative results.** Tasks where your model fails (like our Belebele results) are scientifically valuable.
4. **Verify results make sense.** Base model should be near chance. If it is not, something is wrong with your evaluation.
5. **Save all results.** Raw outputs, not just summary statistics. You will need them for debugging and paper writing.

Repository Reference

Evaluation scripts: `eval/scripts/`Results: `eval/results/`

11.9 Phase 9: Release (1–2 days)

1. **Upload model weights** to HuggingFace (or equivalent). Include both base and SFT models.
2. **Include the tokenizer.** Users need the exact tokenizer to use your model.
3. **Write a model card.** Document architecture, training data, intended use, limitations, and known biases.
4. **Release code.** All training scripts, data preparation, evaluation code, and configs.
5. **Include results.** Publish evaluation results so others can compare.

Repository Reference

Complete repository: <https://github.com/fatherRonnen/semitic-gpt>
weights: <https://huggingface.co/Slasky/SemiticGPT-3B>

Model

11.10 Complete Timeline

Phase	Activity	Duration	GPU Needed?
1	Define scope	1–2 days	No
2	Build tokenizer	1–2 days	No
3	Collect & prepare data	1–2 weeks	No
4	Proxy experiments	2–3 days	1× L40S/A10G
5	Pretraining	1–3 days	8× H100
6	Prepare SFT data	2–3 days	No
7	Supervised fine-tuning	1 day	1× A10G
8	Evaluation	1–2 days	1× A10G
9	Release	1–2 days	No
Total		3–5 weeks	

Table 21: Realistic timeline for building a multilingual model from scratch. Data collection dominates; pretraining is actually one of the shorter phases.

11.11 The Most Important Advice

If we could give only three pieces of advice to someone building their first multilingual model:

1. **Invest in data quality over data quantity.** A smaller clean dataset beats a larger noisy one. Spend the first week on data, not GPUs.
2. **Verify everything.** Labels, splits, file sizes, instance types, model classes. The 10 minutes of checking saves days of debugging. Run the verification checklist from Chapter 10 before every training job.
3. **Test at small scale first.** Proxy experiments cost 1% of the full run. Use them. The recipe that works at 110M will probably work at 3B. The recipe that fails at 110M will definitely fail at 3B.

Final Words

We built SemiticGPT—a 3.14-billion parameter model covering four languages in three scripts—on a single compute node in under 23 hours, using publicly available data and open-source tools. The model demonstrates genuine cross-lingual capabilities: shared representations emerge from multilingual pretraining, and targeted fine-tuning unlocks task performance.

More broadly, our experience suggests that for smaller multilingual models, shared language representations can emerge during pretraining, but usable cross-lingual task performance still depends on explicit multilingual supervision.

We hope this guide demystifies the process of building language models and encourages others to build models for their own under-served language communities. The tools exist. The data, while limited, is findable. The main barrier is knowledge—and that is what this guide is for.

All code, data pipelines, model weights, and evaluation scripts are available at:

<https://github.com/fatherRonnen/semitic-gpt>
<https://huggingface.co/Slasky/SemiticGPT-3B>

References

- A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *NeurIPS*, 2017.
- J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*, 2019.
- A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever. Improving language understanding by generative pre-training. OpenAI Technical Report, 2018.
- A. Radford, J. Wu, R. Child, et al. Language models are unsupervised multitask learners. OpenAI Technical Report, 2019.
- T. Brown, B. Mann, N. Ryder, et al. Language models are few-shot learners. In *NeurIPS*, 2020.
- C. Raffel, N. Shazeer, A. Roberts, et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *JMLR*, 2020.
- R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. In *ACL*, 2016.
- T. Kudo and J. Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *EMNLP*, 2018.
- J. Su, Y. Lu, S. Pan, et al. RoFormer: Enhanced transformer with rotary position embedding. *arXiv:2104.09864*, 2021.
- N. Shazeer. GLU variants improve transformer. *arXiv:2002.05202*, 2020.
- B. Zhang and R. Sennrich. Root mean square layer normalization. In *NeurIPS*, 2019.
- A. Conneau, K. Khandelwal, N. Goyal, et al. Unsupervised cross-lingual representation learning at scale. In *Proc. ACL*, 2020.
- N. Sengupta et al. Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv:2308.16149*, 2023.
- W. Antoun, F. Baly, and H. Hajj. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proc. WANLP*, 2021.
- L. Bandarkar et al. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. *arXiv:2308.16884*, 2023.
- L.B. Allal et al. SmolLM3: The complete blueprint of a state-of-the-art 3B parameter LLM. HuggingFace Technical Report, 2025.
- A. Karpathy. autoresearch: Autonomous AI-driven ML experimentation. GitHub repository, 2026.
- R. Slasky. Autonomous AI-driven Hebrew language model optimization: From random search to optimizer-architecture co-discovery. Independent research report, 2026.
- B. Zhang, P. Williams, I. Titov, and R. Sennrich. Improving massively multilingual neural machine translation and zero-shot translation. In *Proc. ACL*, 2020.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- Y. Zhao et al. PyTorch FSDP: Experiences on scaling fully sharded data parallel. In *VLDB*, 2023.